
CSC458

Computer Networking Systems

Last updated December 24, 2025

Contents

1	Delivering Packets	5
1.1	Layering in the internet	6
1.2	How to transfer a file	7
1.3	Packet Switching and Circuit switching	7
2	Link Layer	8
2.1	Framing	8
2.2	Mac Address	9
2.3	IP Addresses	9
2.4	Model of a Link / Calculating Delays	9
2.4.1	Example Calculations	10
2.5	Round Trip Time	11
2.6	Throughput	11
2.7	Data-In-Flight	11
2.8	Packet Switching	12
2.9	Store-And-Forward vs. Cut Through Switching	13
2.10	Queueing Delay	13
2.11	Increasing Link Bandwidth	14
3	Error Detection and Correction	14
3.1	Ways to Correct Errors	14
3.2	Attempts to Fix or Detect Errors	14
3.3	Encoding to Detect Errors	15
3.3.1	Hamming Distance	15
3.3.2	Hamming Distance Theorems	15
3.3.3	Parity Bits	16
3.4	Checksums	17
4	Interconnecting LANs	17
4.1	Ethernet uses CSMA/CD	17
4.2	Limitations on Ethernet Length	19

4.3	The Ethernet Frame Structure	19
4.4	Link Layers: Bridges	20
4.5	Link Layer: Switches	21
4.6	Self-Learning	22
4.6.1	Flooding can create loops	23
4.6.2	How do we fix the looping problem? (By making Spanning Trees)	23
4.6.3	Constructing a Spanning Tree	24
4.6.4	The Spanning Tree Algorithm	24
4.7	Robust Spanning Tree Algorithm	25
4.8	Spanning Tree Protocol Strengths and Weaknesses	26
5	The IP Layer	26
5.1	Characteristics	26
5.2	The IP Datagram	27
5.3	IP Addresses	27
5.4	Sending Messages	28
5.5	Are Hosts on the network	28
5.5.1	Historical Aside	29
5.6	Subnetting and Supernetting	29
5.7	DNS	30
5.8	ICMP (Internet's error reporting)	30
5.9	Fragmentation	30
5.10	MTU Discovery	32
6	Forwarding	32
6.1	Switches vs. Routers	32
6.2	Longest Prefix Match	33
6.3	Routing Techniques	33
6.4	Forwarding in an IP Router	34
6.5	Distance-Vector Algorithm (Bellman-Ford)	35
6.5.1	Distance-Vector Algorithm Pitfalls	36
6.6	Link-State Packet	37
6.6.1	Calculating Routes and Building a Table	38

6.7	Comparison	39
7	Network Layer LEGACY	40
7.1	Fragmentation	40
7.2	Switches vs. Routers	41
7.3	How does a Router Work?	41
7.4	Forwarding in an IP Router	42
7.5	Organizing routing tables	42
7.6	Longest Prefix Match	43
7.7	Where do Forwarding Tables Come From	43
7.8	Packet Routing And Forwarding	43
7.9	Distance Vector Shortest Path	43
7.10	Link State	44
7.11	Reliable Flooding of LSP	45
8	Internet Topology (Autonomous Systems)	45
8.1	Characteristics of AS paths	47
8.2	Points of Presence	47
8.3	Identifying AS	47
8.4	Customer-Provider Relationship	48
8.5	Peer-Peer Relationship	49
8.6	Things that don't happen	50
8.7	BGP Vs. IGP	51
8.8	Internal Routing Protocols	51
8.9	Routing between Autonomous Systems	52
8.10	BGP Operations	52
8.11	BGP Path Selection	53
8.12	BCP Policy	53
9	TCP / UDP	54
9.1	Types of Transport Protocol	55
9.2	Multiplexing / Demultiplexing	55
9.3	UDP "User Datagram Protocol"	56

9.4	TCP	57
9.5	Sending Data VIA TCP	57
9.6	Sequence Numbers + ISN	58
9.7	TCP 3-Way Handshake	58
9.7.1	When the SYN gets lost	59
9.7.2	Automatic Repeat Request (ARQ)	60
9.7.3	Trade-offs of Timing Out	60
9.8	TCP Sliding Window	61
9.9	Fast Retransmission	62
10	Congestion Control	63
10.1	Congestion Window Evolution	64
10.2	Why are Deep Buffer Routers not good?	66
10.3	Can't Change these	66
10.4	Repeating Slow Start After Idle Period	67
10.5	Nagle's Algorithm	67
10.6	Motivation for Delayed ACK	68
10.6.1	Where does delayed ACK come to play?	68
11	DCN	69
11.1	Data Center Network	69
11.2	How did data centers evolve?	70
11.3	Data Center Network Design Dimensions	70
11.4	Architecture of a Data Center Network	71
11.4.1	Routing Traffic Within a rack	71
11.5	Other data center network topologies	73
11.6	Data center properties	74
11.6.1	ECN+DCTCP	74
11.7	Link Layer Flow Control	75
11.7.1	Proactive	75
11.7.2	Pause-Based	75
11.8	Remote Direct memory Access	76

12 Networks used in machine learning	77
12.1 Moore's law limitation	78
12.2 What makes ML different?	79
12.3 Load-balancing	79
12.4	79
12.5 Incast Problem	81
12.6 Network Aware Applications	82
12.7 ML to enhance computer networks	83
13 Overlay Networks	83
13.1 Tunneling in other layers	84
13.2 What is TOR?	85
13.3 Communication with Mobile Users	85
13.4	86
13.5 Resilient Overlay Network	86
13.6 P2P	86
13.7 Bit-you-know-what-this-is	89

1 Delivering Packets

Sending large data over the internet breaks them in packets, which are sent over the internet to the recipient (well, at least the admin of the recipient as the recipient is not supposed to know that this happens).

When you send something, you can get an estimate when a packet is supposed to be delivered, however

- There are no guarantees that packages will be delivered
- If so, packets are not guaranteed to be in order

Since packet sends can fail:

- Receiver needs to acknowledge receives (positive – got it, negative – didn't get it)

- Needs to keep local copies
- What if an acknowledgement is lost?

Moreover, with the internet:

- Data integrity is not guaranteed. Random bits can flip.
- Packets can be fragmented. This is a serious possibility. What if a packet reaches a network with a smaller packet size limitation?
 - E.g. you send a 9KB packet through a network that can't have packets that large, it results in fragmentation
- Packets may be duplicated. The network may end up duplicating the packets you are sending.

1.1 Layering in the internet

- Application layer
 - You or the application you run do this
- Transport layer
 - Admin / Network / Operating system
 - On behalf of the application, carries the responsibility of providing reliable, in-sequence, E2E delivery of data.
- Network layer
 - Provide the best effort service (analogue to the postal service). No sequencing and reliability, just sends the packages from one side of the globe to the other side of the globe
 - Packets may go through multiple network nodes
- Link layer
 - Packet is injected on one side of the link, ejected from the other side of the link. Analogue to actual postal office trucks.

1.2 How to transfer a file

1. FTP application asks through an **API** (this is socket programming), you make certain calls to the OS that you want to send data to someone else in the network. A desire to send something to host B through a system call.
2. The **TCP** (transmission control protocol) – A TCP connection setup packet is set up. It's a packet to say I want to send something. No content, header is a connection setup header.
3. TCP sends packet to the **IP** layer. The IP receives the TCP packet. It encapsulates the TCP packet in an IP packet. The data portion of the IP packet is the TCP packet, with a newer header: destination address, source address, and protocol.
4. The IP packet is delivered to the link layer. It then puts the packet in the data portion of an ethernet (**MAC**) packet. A new header is created.
5. A router receives the package. It opens up the ethernet packet, and looks at the destination of the IP packet. It then re-encapsulates the IP packet in another ethernet layer and sends it to the next router.
6. This process repeats.
7. ...
8. When the ethernet packet reaches the final router, it is opened up. The receiving host verifies that it is for itself. The transport layer opens the packet. It sees that there is a connection setup request. And then it sends back an acknowledgement.

Each layer takes something, puts it in another envelope, then sends it. Does it belongs to me? Open it up. Does not? Keep sending it.

Layers talk to each other. It's just layers below it does the heavy lifting.

1.3 Packet Switching and Circuit switching

Early on, for phone systems, we had circuit-switched systems. We have a network, with a source A who wants to communicate with a destination B. It's the method used by

the telephone network. Circuit switching dedicates a path for you. That's too old and therefore expensive, since the path is dedicated to you.

Packet switching is used by the internet. Each packet is individually routed, every piece is put in a packet so it can find its way to the destination. It is the method that makes the internet possible.

2 Link Layer

Media is what is used to send bytes from one place to another. Can either be:

- Wire
- Fiber
- Wireless

In the physical layer, you encode bytes, and to send bytes from one adapter to another, you'll have to send them over and generate an analogue waveform from digital data at a transmitter and the receiver must be able to recover it. There are different ways of mapping bits to signals.

2.1 Framing

We can't just send garbage bits, since the receiver gets bits all the time. We need to signal the receiver that they are getting useful information. This is done by framing.

When you send a packet, it is put into a frame before the packet is sent off. The frame consists of:

- The sentinel 0x7E
- The actual frame content
- The ending sentinel 0x7F

To prevent the ending sentinel from being injected, replace all instances of 0x7F with 0x7FX... there will always be a way.

```
1  7E  ...  7F
```

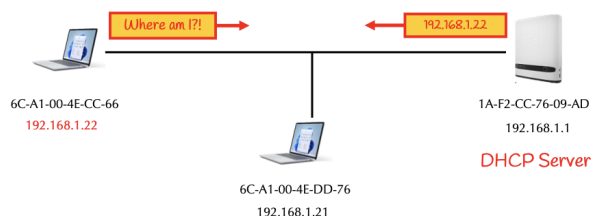
2.2 Mac Address

All your adaptors are built with unique mac addresses, which for the purposes of this course, assume that they cannot be changed. It is a flat name space of 48 bits, in the form AA-BB-CC-DD-EE-FF in hex. It works like a social insurance number.

The first 24 bits of a mac address are usually assigned by vendors (e.g. Dell) by the IEEE. The last 24 bits are assigned by the vendor from its block. There is a broadcast mac address, which requests that a frame be sent to **all** adapters.

2.3 IP Addresses

IP addresses can change. Let's assume, for now, that your only method of connection is through a wire and these are the only three devices connected by that wire.



If on the same network, there is an ARP table that maps an IP address to the mac address. That's how you know someone's mac address.

2.4 Model of a Link / Calculating Delays

You want to send:

- An M Bit Message

It reaches the recipient

- At a rate of R bits per second
- At a delay of D seconds

So, how long does it take to send a message?

Firstly, we need to look at the delays:

- **Propagation delay (D)**

- Propagation delay = $\frac{\text{Distance}}{\text{speed of light in media}}$

- **Transmission delay (M/R)**

- $\frac{M}{R}$ (message size / rate)
 - How quickly can you inject the message on the wire. In other words, it takes this long for the message to come out on the other side of the wire.
 - Unit analysis: $\frac{[\text{bits}]}{[\text{bits}][\text{seconds}^{-1}]} = \text{seconds}$

- **Queueing delay**

So therefore, we get:

$$\text{LATENCY} = \text{Propagation delay} + \text{Transmit delay} + \text{Queue delay}$$

2.4.1 Example Calculations

If, for now, we neglect queueing delay, then as long as we can calculate the propagation and transmission delay, then we can calculate the latency. For example:

If we have $D = 0.010$ s, $R = 56000$ b, $M = 8000$ b, then latency = $0.010 + \frac{8000}{56000}$ = 0.153 s

transmission delay

If we have a T3 (45Mbps) line, we have $D = 0.050$, $R = 45 \times 10^6$, $M = 8000$, then latency = $0.050 + \frac{8000}{45 \times 10^6} = 0.050$ milliseconds – in this case, since the rate is so high then we only suffer the propagation delay D .

2.5 Round Trip Time

Latency is typically the one-way delay over a link. A round trip is the time that holds sending a signal and getting it back. Therefore, the round-trip time (RTT) is **twice the one-way delay**.

2.6 Throughput

The measure of the system's ability to "pump out" data. This is *not* the same as bandwidth.

$$\text{Throughput} = \frac{\text{Transfer size}}{\text{Transfer time}}$$

This is **not** the same as bandwidth. It's usually lower than bandwidth, despite having the same units, since it has to account for more things. Note that:

- Transfer time = sum of
 - Time to get "started shipping the bits" (new thing)
 - Time to ship the bits (your typical bandwidth)
 - Time to get "stopped shipping the bits" (new)

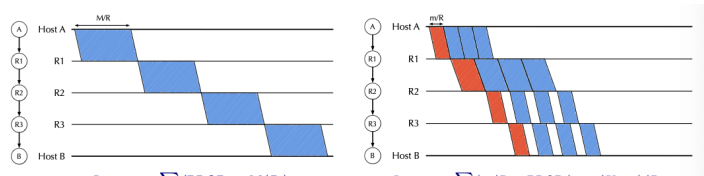
2.7 Data-In-Flight

Consider a 1 bit per second network. Suppose that latency is 16 seconds. Therefore, the network, or moreover "data in flight" is up to **1 b/s × 16s = 16b**. This is known as the bandwidth-delay product. Therefore, we need to have sent 16 bytes (if we keep sending) before we can hear back from the receiver.

2.8 Packet Switching

When data is sent from a source to a destination, it might go through multiple routers. It takes time for data to be given to a router before the router can transmit the data to the next one.

If R_i is the rate of which router i sends data, take a look at this graph ($R_{\min} = \min_i R_i$):



Without parallelizing the sends, the latency is

$$\sum_i \left(\text{PROP}_i + \frac{M}{R_i} \right)$$

However, with parallelizing the sends, it ends up being

$$\begin{aligned} & \sum_i \left(\frac{m}{R_i} + \text{PROP}_i \right) + \frac{m(K-1)}{R_{\min}} \\ &= \left(m \sum_i \frac{1}{R_i} \right) + \left(\sum_i \text{PROP}_i \right) + \left(\frac{m(K-1)}{\min_i R_i} \right) \end{aligned}$$

Where M is the message size, m is the packet size $\frac{M}{\text{packet count}}$, and K is the number of packets in that message.

Breaking messages into packets allow for parallel transmission across all links, reducing end-to-end latency. It prevents a link from being “hogged” for a long time by one message.

If you look at this equation:

$$\underbrace{\left(m \sum_i \frac{1}{R_i}\right)}_1 + \underbrace{\left(\sum_i \text{PROP}_i\right)}_2 + \underbrace{\left(\frac{m(K-1)}{\min_i R_i}\right)}_3$$

- Term 1 is the width of each individual trapezoid.
- Term 2 is the skew of each individual trapezoid.
- Term 3 accounts for the bottleneck.

2.9 Store-And-Forward vs. Cut Through Switching

You'll know that for store and forward, it's going to be

$$\sum \text{PROP} + \sum \text{TRANSMIT}$$

Through all hops.

For cut-through switching, it's

$$\sum \text{PROP} + \max(\text{TRANSMIT})$$

Focus on the bottleneck. And the things that can't be made any faster.

2.10 Queueing Delay

Because the egress link is not always free when a packet arrives, it might be queued in a buffer. If the network is busy, packets might take forever.

And you have no way of computing the queueing delay, because it depends on user behavior.

$$\text{E2E latency} = \sum_i (\text{TRANSPORT}_i + \text{PROP}_i + Q_i)$$

2.11 Increasing Link Bandwidth

- Increasing wire count
- Increasing bits per second

3 Error Detection and Correction

How do we detect and correct messages that don't make it past transmission? The responsibility for making these cuts across the different layers. What happens when a bit gets flipped?

3.1 Ways to Correct Errors

- Detect and retransmit (ARQ)
 - Less expensive unless too many packets have errors
 - (Low overhead, less and simpler processing)
- Error correcting codes
 - Transmitting these are more expensive, but cheaper if errors do occur
 - Reduces the number of retransmissions

Most data networks today use error detection, not error correction. Being said:

- If errors are frequent but small, go for correction.
- If a single message tends to get corrupted by a lot, go for error detection.

3.2 Attempts to Fix or Detect Errors

We can send two copies of the data.

If each “copy” is different, we know there's an error.

But we don't know how to fix it.

Being said, if we send three copies of the data, as long as the bit flips are not too much, we can correct errors.

3.3 Encoding to Detect Errors

Of course, this requires understanding from both the sender and the receiver. You can re-encode your message with redundancy such that your message can be checked and potentially corrected for errors. The simplest way, as stated before, is by duplicating the message. That might not be the most effective. At least, here are some new concepts.

Errors must not turn one valid codeword into another valid codeword, or we cannot detect them

3.3.1 Hamming Distance

The hamming distance of a code is the smallest number of bit differences that turn any one codeword to another

If a code consists of a bunch of codewords, it's the hamming distance of the codeword with the lowest hamming distance.

3.3.2 Hamming Distance Theorems

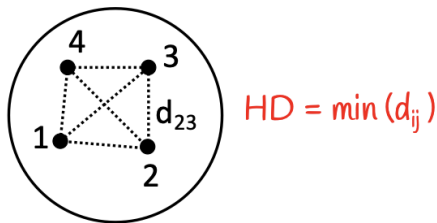
For a code with distance at least $d + 1$, d errors can be detected.

Proof: $c_1 \xrightarrow{d} c'_1 \xrightarrow{1} c'_2 \xrightarrow{d} c_2$ in the worst case.

For a code with distance at least $2d + 1$, d errors can be corrected.

Proof: $c_1 \xrightarrow{d} c'_1 \xrightarrow{\text{at least } d+1} c_2$ in the worst case.

Set of **codes**

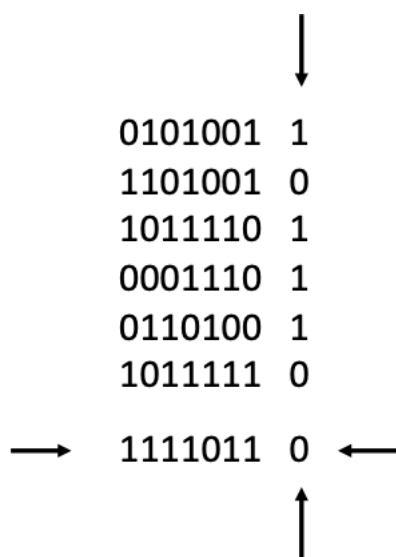


To reliably **detect** a d -bit error: $HD \geq d+1$

To reliably **correct** a d -bit error: $HD \geq 2d+1$

3.3.3 Parity Bits

Start with n bits and add another so that the total number of 1s is even. This can be done consistently. E.g. 0110010 $\rightarrow$ 01100101. This will detect an odd number of bit errors, but not an even one. This does not correct any errors. Can this be improved? By having 2D parity.



This one detects all 1, 2, 3 bit errors and many errors with > 3 bits, and corrects all 1-bit errors. The new numbers ensure that each column and row have an even number of 1s.

3.4 Checksums

No, your checksum does not need to be explicit. The idea of checksums is for you to add up all your data, then send the negative of it along with your data. Explicitly:

- The **checksum** is the 1s complement (negative) of the 1s complement sum of the data

Where:

- 1s complement flip the bits
- 1s complement sum: regular bitwise sum, but if there's an overflow carry-on, wrap it to the back (to the LSB).
 - I'm presuming this halts.

When your checksum is included, when the recipient sums up all the data, it should sum to 0.

4 Interconnecting LANs

Instead of having two devices, what if we have many devices sharing the same medium. The most dominant technology is ethernet. It has been one of the most successful protocols that was designed to connect machines in a local area network. It has both wireless and wired connections, supports a range of speeds (from 10Mbps to 400Gbps), and maybe way more over one connection.

There are multiple devices connecting over a shared medium. That creates some problems.

4.1 Ethernet uses CSMA/CD

So, we have the same medium used across different devices. Let's call the space the **collision domain**. If two devices try to talk at the same time, there will be a collision in

the communication. In a shared medium, only once device can talk at a time. What does ethernet do?

- Carrier sense: Wait until link to be idle.
 - You can start talking as the link is idle.
 - Channel idle: start transmitting
 - Channel busy: wait until idle
 - This method gets to live due to its simplicity.
- Collision detection
 - What happens when two devices talk at the same time?
 - While you're talking, you will listen to the medium as well. If you detect a collision, pause, stop, wait for a random time, and then send a jam signal (some noise that ensures the other side knows a collision occurs).
 - If no collisions occur, then the transmission can complete as normal.
- After a collision
 - After a collision, wait for a random time before trying again.
 - After the m th collision, choose K randomly between 0 to $2^m - 1$ and wait for $K \times 512$ bit times (multiply by the time it takes to transmit 512 bits) before trying again.
 - m get to reset to 0 after transmitting something without a collision

This is a **simple** but very effective solution. You don't have to add complex systems. This runs on the assumption that none of the nodes are malicious.

There are reasons behind every single decision that has been made. The fact that you have to wait for $K \times 512$, 512 is the minimum frame size for ethernet. You don't want to go below that. But that creates some interesting problems.

4.2 Limitations on Ethernet Length

Suppose A sends a message to B . How long does it take? The latency of this link is D seconds. How long does it take for the message to get from A to B :

- At time 0, A sends the message.
- If B is silent at time D , then B hears that A is talking.
 - If, at time $D - \epsilon$, B has not heard from A , but B starts talking. ϵ seconds later, B realizes there is a collision. A doesn't know. That's why B is going to send a jam signal (noise) to make A realize that there was a collision.
 - The Jam signal returns to A at time $2D$. At the time $2D$, A realizes when there is a collision.
- A 's transmission is successful after time $2D$ when the message was sent.
- There is a cap on the size of ethernet connection. If the minimum size of the packet is 512 bits, 25000 meters is the maximum length of the wire.

4.3 The Ethernet Frame Structure

Destination comes before source because the destination is more important than the source. The structure is as follows:

- Preamble
- Destination address
- Source address
- Type
 - Indicates the higher layer protocol (usually IP)
- Data
- CRC
 - Cyclic redundancy test

Ethernet is a connectionless service. No handshaking is required. You just send the message.

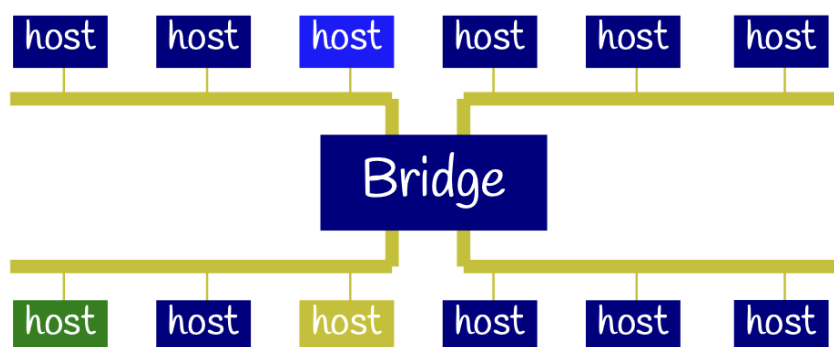
Ethernet is unreliable. Receiving adapter doesn't send ACKs or NACKs.

Packets passed to the network layer may have gaps. Gaps will be filled if application is using TCP. Otherwise, application will see the gaps.

4.4 Link Layers: Bridges

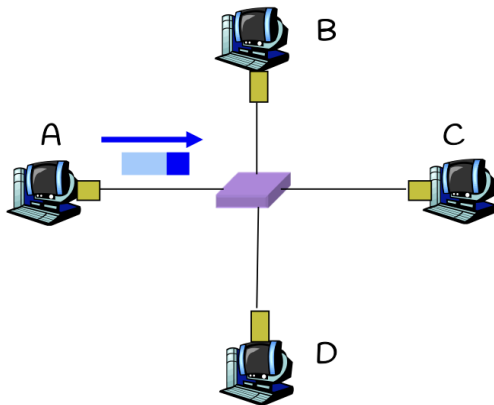
Ethernet medium can't be too long. A single ethernet network is a collision domain, where two devices cannot talk at the same time.

When you have a **bridge**, the job of the bridge is to transfer packets to another network if the receiver happens to be on the other side of the network. The bridge can be over more than one ethernet network. Each segment is in its own collision domain, and the bridge only delivers messages if the other side is idle.



This breaks up collision domains, allowing the network to be expanded.

4.5 Link Layer: Switches



Switches connect individual computers. Essentially the same as a bridge... not quite.

- Dedicated access: Host has a direct connection to the switch rather than a shared LAN connection.
- Full duplex
 - Each connection can be sent in both directions
- Completely avoids collisions.

Buffering the data stored in switches takes time. Buffering delay can be a high fraction of the total delay. To solve this problem, use cut-through switching. As soon as header received, keep transmitting when the outgoing link is idle.

But what happens if links don't have the same rate? You can't transmit something you haven't received, so if the in link has the same or faster rate than the out rate.

The ethernet packet has a CRC field that checks for errors. These are issues to be dealt with, but if you fix these problems using cut-through switching, it solves a lot of the problems.

4.6 Self-Learning

The switch stores a table mapping ports to device. That table starts of empty, so the switch does not know what every port maps to.

Whenever a switch receives a packet:

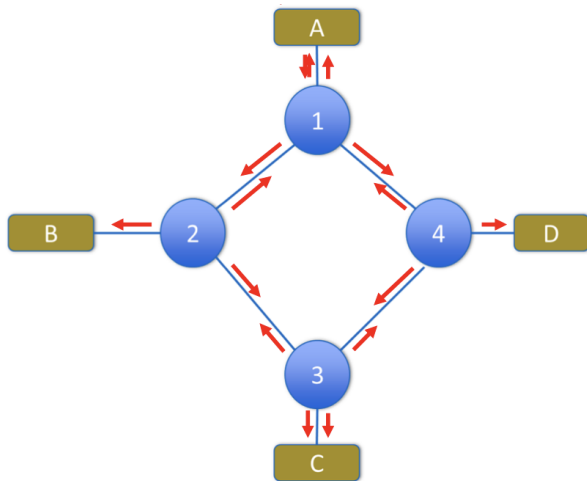
- When A sends a packet to the switch, the switch learns where A is.
- If the switch has to forward a packet to a destination it does not know, if has to flood by sending it to everyone.
- **ABSOLUTELY NOTHING MORE, NOT EVEN THE RECIPIENT.** If the recipient doesn't do anything, well that's the most wasteful thing that could happen.

Precisely:

- When a switch receives a frame
- If entry found for destination then
 - If destination is on the segment from which frame arrived
 - * Then drop the frame
 - * Else forward the frame on interface indicated
- Else flood

Flooding does not make me know anything more. Which means:

4.6.1 Flooding can create loops



You get the picture.

4.6.2 How do we fix the looping problem? (By making Spanning Trees)

- Even though we have a network, we should remove some of the links. Since a network can be represented as a graph, if the graph has cycles, packets can rotate forever.
- **So convert the graph into a spanning tree.**

There are multiple spanning trees that can be composed from a graph (there are different ways).

The spanning tree protocol is a protocol by which switches and bridges construct a spanning tree. It has:

- Zero configuration
- Self-healing

Should have some constraints for backwards compatibility.

4.6.3 Constructing a Spanning Tree

We need a distributed algorithm. Switches cooperate to build the spanning tree and adapt automatically when failures occur.

Key ingredients of the algorithm:

- The root. One of the nodes in the graph needs to be the root.
- Each switch must identify the interfaces that are on the shortest path to the root
- Exclude all other interfaces

Nope, not a good algorithm.

4.6.4 The Spanning Tree Algorithm

Each node broadcasts a message (Y, d, X) .

- Proposes Y as a root
- From node X
- Saying that the distance from the root (between Y and X is d)

And this keeps happening:

- Initially, each switch proposes itself as the root
 - For all $X \in \text{Switches}$, X announces $\left(\begin{smallmatrix} X \\ \text{root} \end{smallmatrix}, \begin{smallmatrix} 0 \\ \text{dist} \end{smallmatrix}, \begin{smallmatrix} X \\ \text{self} \end{smallmatrix} \right)$
- Switches update their view of the root and their distance to the root, where if they hear someone else say the root is different, they listen if the broadcasted root's ID is lower.
 - At each switch Z , whenever a message (Y, d, X) is received from X :
 - * If Y 's ID < current root:
 - Root = Y
 - Shortest distance to root = $d + 1$

- * If Y 's ID = current root and shortest distance to root $> \underset{\text{from message}}{d} + 1$
 - Shortest distance to root = $d + 1$
- If root changed or shortest distance to root changed:
 - * Send all neighbors message (Y , shortest dist to root, Z)

Repeat until no change. Identify interfaces (links) not on a shortest path to the root and exclude them from the spanning tree.

Sometimes, there is a tie in the algorithm. When there are multiple shortest paths to the root (e.g. node 4 and node 5 have the same distance to the root), choose the path to the neighbor that has the smallest identifier.

This algorithm converges, but there are potential things we need to resolve.

- What if the root node or any other switch in the node fails?
 - Need to elect a new root. This time, use the robust spanning tree algorithm.

The algorithm must react to failures.

4.7 Robust Spanning Tree Algorithm

Root switch continues sending messages, periodically reannouncing itself as the root. Other switches continue forwarding messages.

Errors are detected through timeout (soft state). Switches wait to hear from others, and eventually times out and claims to be the root.

This works if a link or any other part of the network goes down. This message gets sent to all of the neighbors of the node, to all edges regardless of whether it's active or not.

30 seconds is an arbitrary number. It can be 30 seconds, 3 seconds, 0.3 seconds. The overhead and the reliability changes have impacts. If you do this more often, you'll get faster convergence but more overhead.

4.8 Spanning Tree Protocol Strengths and Weaknesses

Strengths

- Plug n' play: zero configuration
- Simple
- Cheap

Weaknesses:

- Wasted bandwidth (especially the default 30 second delay of the root broadcasting)
- Delay in reestablishing spanning tree
- Slow to react to host movement, need timeout
- Poor predictability: depending on location of root and traffic pattern

This algorithm is not optimal in any sense. It can give longer paths depending on node IDs and how the algorithm works.

5 The IP Layer

The internet protocol, in the layer above the link layer: the network layer.

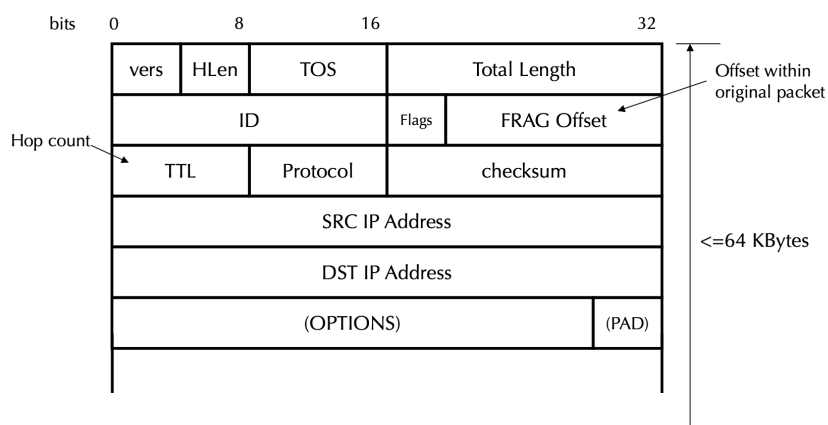
5.1 Characteristics

Like ethernet, IP is connectionless. You are not establishing connection between sender and receiver. You just send it. As long as you have packets, you send them. So:

- Connectionless: there can be mis-sequencing
 - It receives individual packets and delivers packets, but it never is the one that establishes the path between the source and destination.
- Unreliable: may drop packets

- Best efforts: but only if necessary. Only drops if it absolutely has to.
- Datagram: individually routed
 - Going to put everything inside the envelope

5.2 The IP Datagram



Each ward (row) is 32 bits. **HLen** is the row count. Max. is 60 bytes for the header length.

The maximum, IP packet size is $2^{16} = 65536$

5.3 IP Addresses

IPV4 addresses are 32 bits long, usually represented in dotted decimal notation. Every interface has a unique IP address. IP addresses are not linked to devices.

Hence:

- A computer might have two or more IP addresses. A computer can connect to more than one network.
- A router has many IP addresses.

IP addresses are assigned statically or dynamically.

IPV6 addresses are 128 bits long.

5.4 Sending Messages

If we want to send packets from A to B:

- Same network:
 - No need to route, *A* can directly send the packet over the link layer to *B*
- Different network:
 - *A* must send the packet to a router (which has a different IP) that can forward it towards *B*

These scenarios are different!!

5.5 Are Hosts on the network

Every IP address is divided into two parts:

- The network ID
- The host ID

The network ID can be 8 bits, and the host ID can be 24 bits. Or they can be equally split.

If I know my network ID is 1.2.3, then if I have the destination ID, I can compare the start of the address. Therefore I can know that the person I'm talking to is in my network.

How do I know which part of my IP address is the network ID or not? I have to be told that. When I configure my network or get my IP address from DHCP, I get a network mask. It tells me how many bits is in the ID. For example:

- If my IP address is 1.2.3.4/8, it means that the first 8 bits in my IP address is the network. I could also be given a bitwise mask. Therefore taking the & of this, with my IP address, I get the network ID.
- This means I can easily check if someone else's IP address matches my network ID.

This means I can have anything from 1 to 31 bits of network ID. My network mask can change this.

5.5.1 Historical Aside

In early days of the internet, instead of doing this, they didn't want to share the network mask, instead they want everything to be self-contained. The way they did it, is to define different classes of addresses.

- Any IP address that starts with 0 is considered class "A", and these addresses have 8 bits of network ID followed by 24 bits of HostID. This means you can only have 128 class A network... that does not sound like a lot. You'd run out of them in an instant.
- And so on for class B... the entire country of China would have less IP addresses than MIT.

Then people figured out that this didn't work. So, instead of storing everything in the IP address, how about let the network be identified by the network mask?

5.6 Subnetting and Supernetting

A network can allocate more bits at the front to make new sub-networks. If UofT has 8 bits for its network ID, it can break its network into sub networks:

- Network ID + 00 is the CS department
- Network + 01 is the ECE department
- And so on

We can also Supernet. This can be the reverse. Nobody else needs to care about the subnetting that occurs in UofT addresses. You only need to know UofT's network ID.

Subnetting and supernetting has been very helpful – they allow hiding details of internal network from the rest of the world.

5.7 DNS

Domain names are hierarchal.

TLDs are assigned by the internet corporation for assigned names and number (ICANN).

If you have an application and you want the IP address of www.eecs.berkeley.edu, your operating system is going to connect to a local DNS server, asking for the local IP address associated with the URL. If the local DNS does not have that information, it will forward you to the root domain name server (.edu), which then forwards you to berkeley.edu, which then forwards you to eecs.berkeley.edu

Berkely's DNS is responsible for anything inside Berkely.

5.8 ICMP (Internet's error reporting)

When something goes wrong, the router can provide some information. If a packet gets dropped, information can be provided to the end user. We use that information to do something very interesting. About time to live: if a packet's TTL reaches 0, usually, routers that drop the packet saying TTL reaches 0 sends the message back.

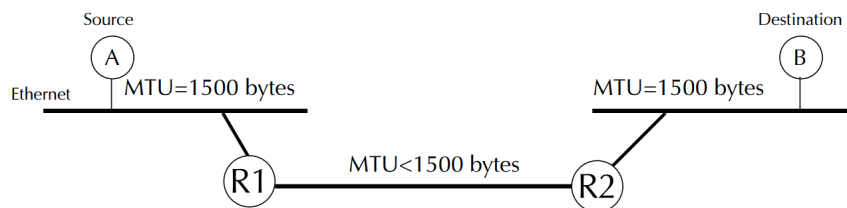
So, if you want to send a packet to Google: sending it to the first router is, that router will drop the packet and send it back. By doing this, you can learn all the router your messages go through. This is exactly what **traceroute** does. Sends a packet w/ TTL1, TTL2, and so on.

An ICMP message is an IP datagram and sends it back to the message sender.

5.9 Fragmentation

On the IP side:

Obviously data you send has to pass through multiple routers.



Links between routers have a maximum transmission unit (**MTU**). This limits the size of packets. Can't send individual IP packets larger than the limit. Though in practice, these tend to be ≥ 1500 bytes.

So, if you need to send something from *A* to *B*, well, it might be routed as follows:

$$A \xrightarrow{\text{MTU}=1500} R_1 \xrightarrow{\text{MTU}=500} R_2 \xrightarrow{\text{MTU}=1500} B$$

So, let's send a packet of size 1500 bytes (for the purposes of simplicity, let's ignore the fact that packets need to be a size that is a multiple of 8, and let's assume header doesn't take any space). It can be sent to R_1 without the need to be fragmented. But the link from R_1 to R_2 can only support packets of 500 bytes.

What do you do? **Fragment it.** Split the IP datagram. Your datagram is initially 1500, it can be split into 3. Under the assumption that headers don't take space (this assumption is wrong), you can get the data across. But this also means having to put more information in the headers so that the fragmented datagrams can be reconstructed.

These two pieces of information are enough for each piece:

- **Offset:** satisfies `data[offset:offset+frag_len]` is what was sent. (`frag_len` is the size of the fragment, after all).
- **More frag:** 1 if this isn't the end of the fragment. 0 if it is. Hence, 1 for all but the last.

There couldn't be anything but the offset. Trying to store the fragment number brings issues if you need to fragment the data again, which **will happen** if somehow your next link can only serve packets even smaller.

In practice, offsets must be a multiple of 8 (during a test, just say you're ignoring this – no need to focus on the hard detail). While the example of $1500 \rightarrow 500$ has

nice numbers, in the case where headers do take up space, splitting up the data just becomes a bit messier just to me.

5.10 MTU Discovery

If you know the destination (can be a URL), you can figure out, across all links in the path, the largest size you can make a packet without causing any fragmentation. That's because you can include a flag which disallows fragmenting, having the routers drop the packet when it fragmented it otherwise. Try the command:

```
1 traceroute -F www.uwaterloo.ca 1500
2 traceroute -F www.uwaterloo.ca 1501
```

If the first command succeeds but the second one fails, you know that 1500 is the maximum size of a packet you can send without any fragmentation. On fail, you'll get an ICMP error message.

6 Forwarding

Done by routers, if IP datagrams need to be hopped.

6.1 Switches vs. Routers

Switches have nice properties. If you have many switches and you connect them to each other, they just work. Is it optimal? No. Depending on how you connect it, it can hurt performance, but it works.

They can do fast filtering and forwarding of frames, no matter how you connect them.

However, disadvantages of switches over routers:

- Topology restricted to a spanning tree, which is sub-optimal

- Large networks require large ARP tables. The ARP table grows with the size of the nodes. In data centers, there can be millions of devices, meaning the ARP table can be extremely large. Occasionally, you will have a broadcast. If your network is 1M nodes, you will have 1M broadcast messages throughout the network.
- Instead of doing link layer switching, you do some network layer routing. Routing happens at the network layer, not the link layer. If you are going beyond your local network, you will be using network layer routers.

6.2 Longest Prefix Match

There are too many IP addresses. IPV4 can only have 2^{32} distinct IP addresses. This is why we have network IDs and host IDs. You care more about the network. All devices in one network can be treated as the same, and therefore, if a router knows what network is being targeted they can already know the link to send it from.

Based on the router's routing table, which can be extremely large, the destination IP address must be matched against entries there.

It is typical to consider forwarding to an egress port that has a corresponding IP address entry in the IPv4 table that happens to be the prefix of the IP address we want to target (In form 123.123.123.123/SUBNET_MASK, only consider it w/ the subnet max). What if there are multiple? Match the one with the longest subnet mask. I mean the one that has the most specificity. Too vague? Don't

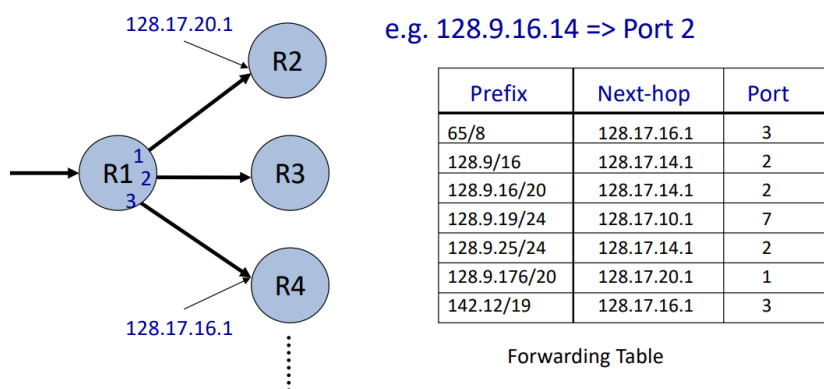
6.3 Routing Techniques

Say we have a bunch of routers in a network with a label (maybe $A, B, C, \dots$), in a network, representable by a graph. Routers are short-sighted, so they can't see all at once. If there isn't a link between A and C , where should A route the data to? If B happens to have a link with A and C , then A can send the data to B so that B can send it to C . Therefore, A should record that whenever it wants to send something to C , it should hop the packet over to B . This will be stored as a table for all possible other routers. How do we build that table?

6.4 Forwarding in an IP Router

Look up packet destination in forwarding table. If known, forward to the correct port. If unknown, drop the packet. Decrement TTL, update header checksum. Forward packet to outgoing interface. Transmit packet onto link.

In a real router, it has to be read through an address book, which probably looks like

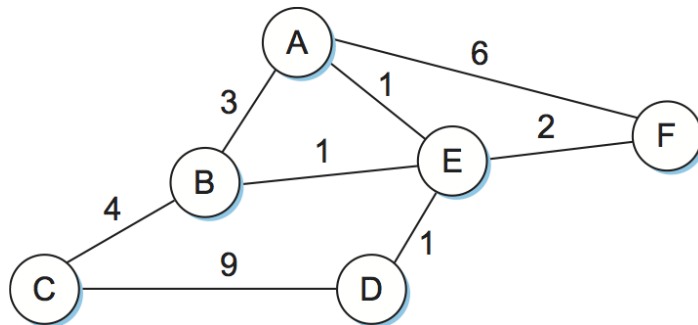


Each router has a forwarding table. It maps destination addresses to outgoing interfaces.

When a packet is received:

- Destination IP address is inspected in the header.
- The table is read, and the outgoing interface is identified
- Packet is forward out of that interface
- Next router takes over

6.5 Distance-Vector Algorithm (Bellman-Ford)



You have a network of routers.

How does each router build up the forwarding table?

The idea is:

- Each node constructs a 1D array containing the distances to all other nodes
- Then each node shares it with its immediate neighbors

Initially each node only knows the cost of its immediate neighbors: for example, C knows that to B, it costs 4, and to D, it costs 9.

	A	B	C	D	E	F	G
A	0	1	1	∞	1	1	∞
B	1	0	1	∞	∞	∞	∞
C	1	1	0	1	∞	∞	∞
D	∞	∞	1	0	∞	∞	1
E	1	∞	∞	∞	0	∞	∞
F	1	∞	∞	∞	∞	0	1
G	∞	∞	∞	1	∞	1	0

Initially, if not as neighbors, cost is set to ∞ .

And then every node sends a message to its directly connected neighbors this list.

Nodes receiving this information update their tables accordingly. For example, B tells C that it costs itself 3 to send to A and 1 to send to E. Thus C learns it costs $4 + 3$ to send

to A from itself and should hop to B , and it costs $4 + 1$ to send to E and should hop to B . Obviously, only update if a cheaper path has been found.

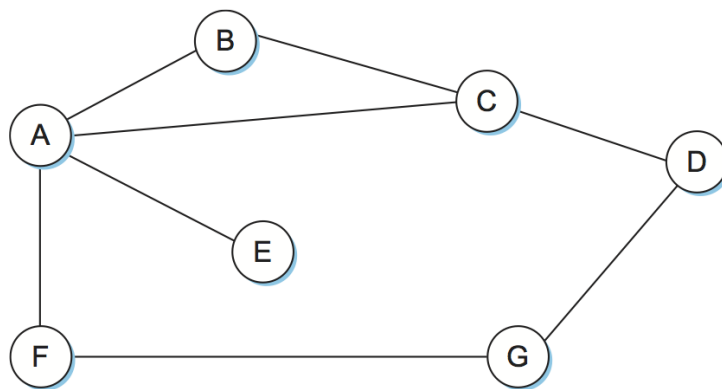
Repeat until convergence.

It is provable that after K iterations, all connections that require K hops or less will be optimal if nothing fails. This is done by proof by induction.

6.5.1 Distance-Vector Algorithm Pitfalls

Fails can happen.

Recall this.



If $A - E$ fails, then A advertises a distance of infinity to E .

But C cannot pick that up. It advertises a distance of 2 to E .

Therefore A thinks that it can use C to get to E , and claims a distance of 3 to E .

This results in the distances constantly increasing until it hits infinity.

So, how to mitigate:

1. Make infinity a small number (16)
2. If C learns from A about E , then C will not teach A (split horizon)
3. If C learns from A about E , then C tells A not to send to it for the purposes of sending to E by advertising a distance of ∞ (split horizon with poison reverse)

6.6 Link-State Packet

Again, we assume that each node is able to find out the state of the link to its neighbors and the cost of each link. Well, we'll have to do flooding, but this time, it's reliable.

Reliable flooding makes sure all participating nodes in the routing protocol get a copy of the link-state information from all the other nodes. Just send out the link-state information out on all its directly connected links. Then each node forwards that information out of all its link. This information is enough for each node to know the entire network.

When each node receives an update packet (link state packet), they'll get the following information in the packet:

- Which node made it?
- List of directly connected neighbors of that node
- A sequence number
- TTL

The first 2 can help reconstruct the network, the last two enable reliability. It ensures that each router always gets the latest and most recent copy of the information, since there can be multiple, contradictory LSPs from one node traversing the network.

When a packet is received, let's say that node X receives a packet from Y :

- X checks to see if it has already stored a copy of an LSP from Y
- If not, the LSP is stored.
- If already, then only store and use the LSP if it has a larger sequence number.
- Afterwards, if stored, LSP is sent to all its neighbors except for where the LSP was sent.

LSPs carry a TTL. This ensures that old link-state information is eventually removed from the network.

Also, if a link goes up and down way too often wait a bit. Maybe 5 seconds.

6.6.1 Calculating Routes and Building a Table

Dijkstra's algorithm is used. For each other node it finds the shortest weighted path. With that it'll know which node to forward the packet to at first.

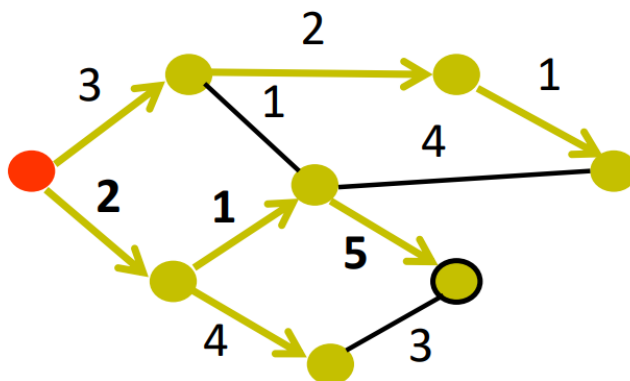
This is the algorithm:

```

1  M = {s}
2  for each n in N - {s}
3      C(n) = l(s,n)
4  while (N != M)
5      M = M + {w} such that C(w) is the minimum for all w in
        (N-M)
6      for each n in (N-M)
7          C(n) = MIN(C(n), C(w)+l(w,n))

```

You'll see that for each node; it builds a spanning tree.



Hence, forwarding table is:

	link
v	(u,v)
w	(u,w)
x	(u,w)
y	(u,v)
z	(u,v)
s	(u,w)
t	(u,w)

6.7 Comparison

	LS (Dijk)	DV
Message complexity	Since flooding happens, $O(nE)$ messages are sent after all. Each node has a message eventually spread through all links.	Exchange only happens through neighbors. Messages aren't flooded. Convergence time varies.
Speed of convergence	$O(n^2)$ algorithm requires $O(nE)$ messages	Convergence time varies. There may be routing loops, and the count-to-infinity problem
Robustness	Node can advertise incorrect link cost. Each node computes only with its own table.	DV node can advertise incorrect path cost. Errors can propagate.

7 Network Layer LEGACY

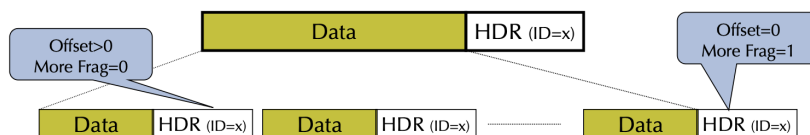
7.1 Fragmentation

A router may receive a packet larger than the maximum transmission unit (MTU) of the outgoing link. Assume that there is a link that has a smaller MTU.

You send a packet, and one of the links have a MTU smaller than your packet. This requires fragmentation. A router will fragment the IP data gram into multiple, self-contained datagrams.

Each fragmented datagram should contain the *offset* (not the fragment count – what if it needs to be fragmented again) and a bit indicated if that is the last fragment.

Solution: R1 fragments the IP datagram into multiple, self-contained datagrams.



Note: offset must be a multiple of 8.

Fragments are re-assembled by the destination host, not by the intermediate routers. If you know the MTU across the path, the host might fragment the data at the start rather than have the routers do the work. This is called **MTU discovery**.

If you are using the same path over and over, sometimes you send different size packets, and see which ones are delivered without fragmentation.

In the IP header, we have a flag called “no fragmentation”. If an IP packet has that bit set to 1, we want to have it not fragment the packet. If the IP router cannot send a packet which wishes not to be fragmented, the packet is dropped. Try:

```
1 traceroute -F www.uwaterloo.ca 1500
2 traceroute -F www.uwaterloo.ca 1501
```

DF=1 set in IP header, routers send ICMP error message, which is shown as: !F

Most links use MTU ≥ 1500

7.2 Switches vs. Routers

Switches have nice properties. If you have many switches and you connect them to each other, they just work. Is it optimal? No. Depending on how you connect it, it can hurt performance, but it works.

They can do fast filtering and forwarding of frames, no matter how you connect them.

However, disadvantages of switches over routers:

- Topology restricted to a spanning tree, which is sub-optimal
- Large networks require large ARP tables. The ARP table grows with the size of the nodes. In data centers, there can be millions of devices, meaning the ARP table can be extremely large. Occasionally, you will have a broadcast. If your network is 1M nodes, you will have 1M broadcast messages throughout the network.
- Instead of doing link layer switching, you do some network layer routing. Routing happens at the network layer, not the link layer. If you are going beyond your local network, you will be using network layer routers.

7.3 How does a Router Work?

Each router has a forwarding table. It:

- Maps destination addresses
- To outgoing interfaces

Packets. Once they arrive in ingress ports, something in the router makes decision for forwarding based on information for the forwarding table. There is a cross-write fabric, based on the decision of the forward table, transports data from the input port to the correct output port. There is a buffer in the router for each output port, which they leave one-by-one.

7.4 Forwarding in an IP Router

- Look up packet DA in forwarding table
 - If known, forward to correct port
 - If unknown, drop packet
- Decrement TTL, update header checksum

Why did we do that? To avoid loops. Occasionally loops are going to be formed. You are talking about a network as big as the internet, and you have way too many entities trying to control the network.

Once TTL updated, update checksum and then forward the packet. Transmit packet onto link.

Look up IP address in router.

7.5 Organizing routing tables

Have entries for IP address. If packet for 1.2.3.4, it somehow knows what egress port to send the data from. And so on. Somehow, it knows this for every destination address.

This works. But it has a problem. What's the problem?

There are too many IP addresses. IPV4 only has 2^{32} IP addresses. Lots of IP addresses. How can we simplify this? Usually, there is a reason we have network IDs and host IDs. Usually, you care about the network. All the devices in one network can be treated the same. So, you create prefixes. If you are an ISP and there are traffic for any of the machine with a specific prefix, just send them over this link. And so on.

That significantly reduces the size of the table, but that creates the problem.

Each IP address matches exactly one IP address per entry. When you have prefixes, some IP addresses might belong to more than one provider.

Instead of performing exact matching for IP addresses, we do the longest prefix match.

7.6 Longest Prefix Match

What does that mean?

Start taking from the start of the IP address, see how many bits we can match, which ever matches more of a prefix should be chosen.

Imagine UofT has a big network with a small prefix. UofT devices its network to multiple subnets, lets say CS has a smaller workload with a larger prexfix. If rogers has two rules, one saying, dero any machine in UofT, send it on port 1, or CS, send it fto port 2, f a packet in CS matches both, then we're going to match 2. There isn;'t any reason t use two rules.

More specific route – longest matching prefix.

7.7 Where do Forwarding Tables Come From

What if your network changes? What if a link goes up or down? We need to have a mechanism that is more than a manual entry of IP prefixes In the routing table?

7.8 Packet Routing And Forwarding

How are routing table entries filled up?

7.9 Distance Vector Shortest Path

After K iterations all K -hop distance paths you should know the most optimal ones.

Iteration for the bellman ford: maximum distance in the network

How do we know the algorithm always converges: mathematical induction. For $k = 1$, we get the most optimal path for 1 hop, and then use induction to prove for all k

What happens if the topology changes or the cost changes?

Well, bad news travels slowly. If a link fails it could result in an infinite loop. Distance-vector protocols suffer from “counting to infinity” problem. It can take very long to converge to optimal path if that happens.

How to mitigate this issue? We won’t solve it entirely.

Solution:

- Change infinity to some small integer (16). If there are 16 iterations and im still not finding the shortest path just say its gone and ignore that path
- Split horizon: okay but can create awkward problems. If R_2 learns the shortest path requires going to R_3 , it will not tell R_3 about anything (not advertising back to R_3)
- Split horizon with poison reverse: Instead of not advertising to R_3 it advertises infinity, to prevent R_3 to sending things back to R_2 .

All of them work in some scenarios but there are corner cases in which some of these techniques break.

7.10 Link State

In distance vector, information is only send to neighbors. Neighbors are the only entities I talk to. In link state protocol, that is not the case. Every node in the network send what it knows about the topology to everyone else in the network.

Flooding is not a good idea. In general it is not agood idea. But link state protoclcs take advantage of flooding in a very limited scale.

If you have a network, and you kno you have 3 links, you flood the network telling everyone in the network you have ___ more links. This information is delivered to everyone throughout the network.

Every node gathers this state information from everyone else. It re-creates the topology of the network, locally. It creates a data structure that represents the topology of the network, which comes from those packets that come from all the node in that network. You now have a local graph, and you know how to find the shortest path in a graph.

Use Dijkstra's single source shortest path algorithm. At each step of the algorithm, router adds the next shortest path to the tree. If you want to send traffic to any other node in the network, how do you route it? Once you run that algorithm, you convert the shortest path information to the routing table.

And by this you make a forwarding table (Dijkstra algorithm constructs a spanning tree – see the slides) that contains the first node in the path that goes to the eventual node you want to go to.

7.11 Reliable Flooding of LSP

The link state packet:

- The ID of the router that created the LSP
- List of directly connected neighbors and cost
- Sequence number
 - This makes sure everyone knows the latest version of the state
- TTL
 - Because we are flooding these things can go in the network forever.

Sometimes, if a link goes up and down way too much, maybe don't rely on it.

8 Internet Topology (Autonomous Systems)

The internet is what is divided into autonomous systems. These are parts of the network that are owned and managed by different institutions. They have their own objectives in managing why and how they do things. The collection of routers and links that are managed by a single entity, could be a company or university, has a bunch of these devices and they are the ones deciding how the network is managed.

We have a hierarchy of autonomous systems. We have a:

- Large tier 1 providers
- With a nationwide network
- Could span across multiple countries or continents

And then we have medium-sized regional providers (rogers, bell). They are not the top tier providers which go over multiple continents

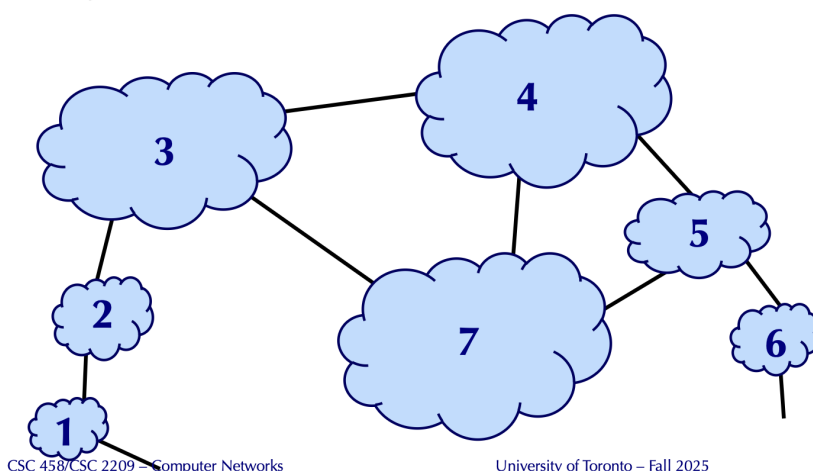
- These medium-sized networks provide service to downstream customers, but they need a provider of their own. This means they typically have a national or regional scope.

And then you have smaller networks managed by individual companies.

These autonomous systems (AS) don't always want to share information to each other. This is what makes life a bit more difficult when it comes to routers.

This new system allows us to create a new abstraction: a graph. Each node is an autonomous system (AS). Each link can be a collection of links that connect these autonomous systems.

Two autonomous systems may have multiple physical links that connect to each other. In the abstraction, we show this as a single link. It is a logical link, not a physical link. Sometimes, there is not even a physical link connecting them. Sometimes, AS starts something called an exchange point, which they all connect to. This creates the opportunity for these AS to connect to each other.



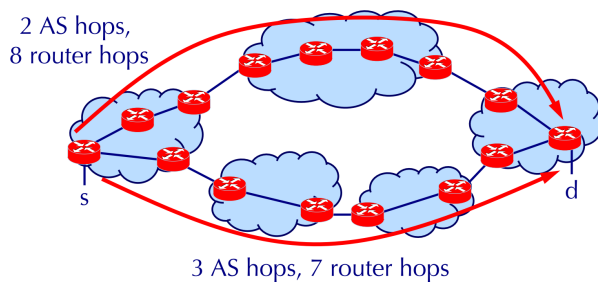
CSC 458/CSC 2209 – Computer Networks

University of Toronto – Fall 2025

7

8.1 Characteristics of AS paths

The AS path may be longer than the shortest path, since the router path may be longer than the shortest path.



8.2 Points of Presence

Points of presence should be placed in high-density areas, or near other providers / exchange points. Of course, beware of cost and real-estate. You'll usually find PoPs in major metropolitan areas.

8.3 Identifying AS

Each AS has a number associated to it. AS numbers are 32-bit values, which used to be 16. Currently estimated to be over 90,000 in use.

Certain companies may have multiple AS, if they cover multiple continents. You don't want to send a packet to the other side of the globe and send it back.

Once you have these AS numbers, you can identify paths. At this level of detail, we don't care about what happens inside the AS. There might be 10s, 100s, 1000s of routers in the AS. But when you are talking about path from a client to a server, server to a client, we talk about going through a path of autonomous systems.

We don't think about the actual routers connect the traffic. We are thinking of which AS are in the way when going from the webserver to the client.

The links can carry extra information, depending on which business relationships these AS have.

The business relationship defines how much traffic the link is supposed to carry. Which destinations to reach. How much money one AS pays to another. There are lots of relationships that exist.

Usually, we classify them to:

- Customer provider
 - Customer pays provider, provider provides connection. Princeton is a customer of AT&T
- Peer-peer
 - Two nodes make an agreement to share each other's customers.
 - Because it's simple.
 - Bell and rogers have a similar no. of customers, so they might reach an agreement.

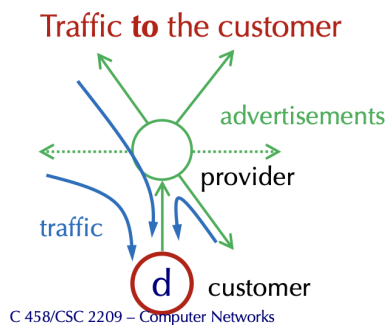
What does it mean to have these kinds of relationships.

8.4 Customer-Provider Relationship

Customer tells the provider what its IP addresses are.

The provider's job is to tell everyone in the world, the IP address in this range are paying me to connect them to the world. When you receive an advertisement, I own, I have a connectivity to the customer d . The provider propagates this to everyone else in the world. If you have traffic to d , I know how to deliver it. It is going to say to its own providers, it can carry traffic to d .

If the customer gets traffic, provider receives the traffic, and the customer gets the traffic.



Customer needs to be reachable for other, so customer needs to be reachable from everyone.

Customer does not want to provide transit service. Customer does not let its provider route through it.

The provider for every IP prefix it knows, it tells the customer it can deliver traffic.

If the customer has traffic to the provider, it will send it to the provider.

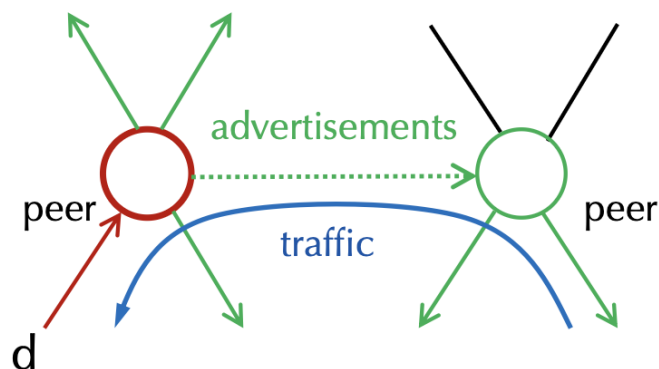
The customer will also send to its own customers, if there are any. This means if the customer needs to send information, maybe from someone else, it sends the information.

For internal networks, like UofT, it is their job to provide service for their own customers. It will not provide the link to other providers.

8.5 Peer-Peer Relationship

If Bell has a customer d , and peer learns about d :

Traffic to/from the peer and its customers



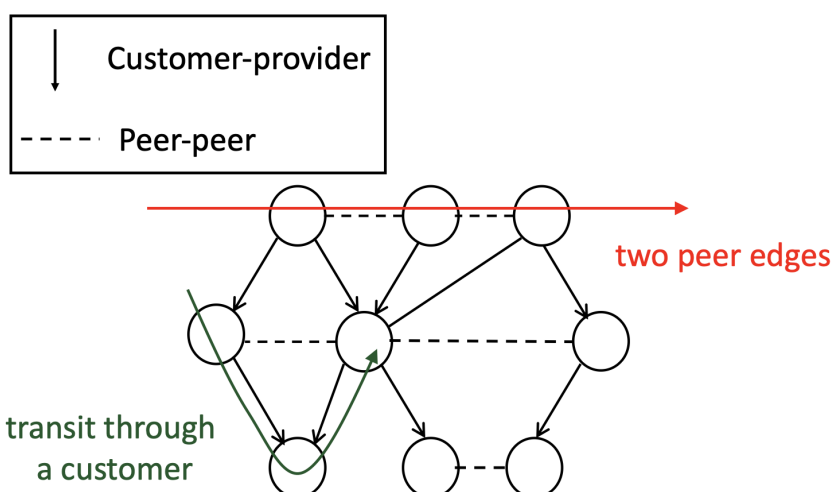
If UofT is a customer of bell, and Waterloo is a customer of rogers, rogers is going to send the advertisement to its customers.

The idea, is AS exports only a peer's routes to its customers.

8.6 Things that don't happen

Having a customer forward something

Paths You Should Never See ("Invalid")



8.7 BGP Vs. IGP

BGP is the border gateway protocol. To send traffic to google, you need to send it to rogers. This should be consistent across all AS.

IGP is the internal gateway protocol. It tells me to go from this node to that border router, the next hop is this. What goes inside is internal to the AS.

When I combine this information: eventually, each router only needs to know where the next hop is. That's it.

8.8 Internal Routing Protocols

- RIP
 - Uses DV
 - Updates sent every 30s
 - No auth
 - Written in BSD
 - Not used
- OSPF
 - Link-state updates sent via flooding
 - Router use Dijkstra's algorithm
 - Authenticated updates
 - AS may be partitioned into areas
 - Widely used

For the AS-level topology:

- Destinations are IP prefixes
- Nodes are AS
- Links are connections and business relationships

8.9 Routing between Autonomous Systems

The path-vector routing between AS is an extension of DV. It supports flexible routing policies and avoids the count-to-infinity problem.

The idea is to advertise the entire path. Hence: to broadcast destination d , which router 1 will do, it can say to router 2: $d: \text{path}(1)$. And then router 2 can say to router 3: $d: \text{path}(2, 1)$. And so on.

Nodes can easily detect a loop, such as it seeing itself in the path.

Nodes can simply discard paths with loops.

This is BGP. It is a path-vector routing protocol. BGP advertises complete paths, saying: “the network 176.64/16 can be reached with the path {AS1, AS5, AS13}. Paths with loops are detected and ignored. When a link or router fails, the path is withdrawn.

8.10 BGP Operations

For two AS to start up a BGP session:

- A session is established on some TCP port
- All active routes are exchanged
- Incremental updates are sent over and over
 - While the connection is ALIVE exchange route UPDATE messages

A node learns multiple paths to the destination. Updates are done incrementally, and if an active route is not available it sends a withdrawal message to all other neighbors

Examples of BGP messages:

- Open
- Keep alive
- Notification: shuts down a peering session
- Update: Announcing new routes or withdrawing previously announced routes

BGP announcement = prefix + path attributes

Where attributes include: next hop, AS Path, local preference, multi-exit discriminator

8.11 BGP Path Selection

In the simplest case, take the shortest AS path and break ties arbitrarily. However, BGP is not limited to shortest path routing. It can do policy-based routing.

So, how do we pick routes:

- Routes learned from the customer are preferred over routes learned from the peer, preferred over route learned from the provider
- The customer advertises if they are **willing** to forward traffic

So with this, this is how BGP route selection goes:

1. Highest local preferences. Prefer customer routes over peer routes.
2. Shortest path, lowest MED, I BGP < E BGP, lowest IGP cost to BGP egress
3. And then break ties by choosing the lowest router ID

YOU SHOULD NEVER SEE A VALLEY!!! It's the customer's responsibility to prevent that from happening

8.12 BCP Policy

Import policy

- Filter unwanted routes from neighbor.
- Assign local preference to favored routes

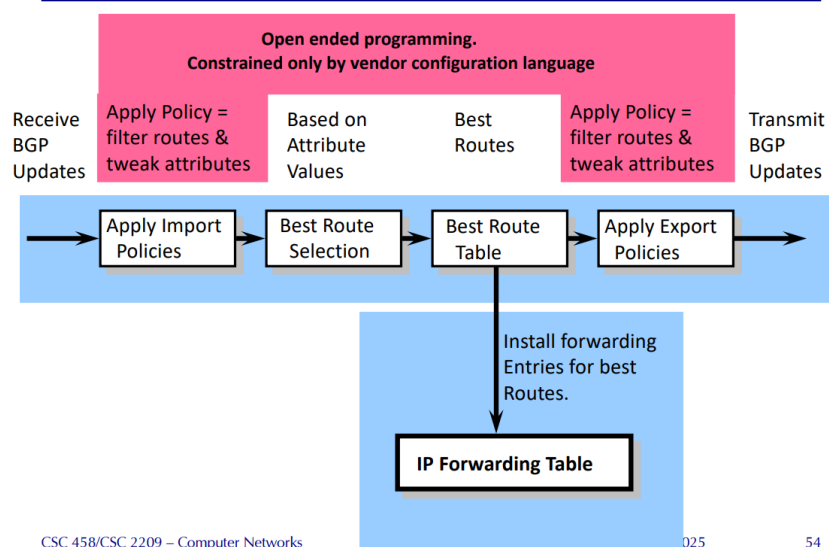
Export policy

- Filter routes you don't want to tell your neighbor

- Manipulate attributes to control what they see, such as making a path artificially longer than it is

If I don't want another router communicating with me or if I want a router to take a specific path, this is something I can do. Lie to my neighbors.

BGP Policy: Influencing Decisions



9 TCP / UDP

Transport layers handle communication between processes, probably at the socket level. It relies on network layer and serves the application layer.

They provide logical communication between application processes running on different hosts. They run on end hosts.

- It's the senders' responsibility to break application messages into segments and pass them onto the network layer
- It's the receiver's responsibility to reassemble these messages and pass them onto the application layer

There are multiple transport protocols available to application. Through the internet, that's TCP and UDP

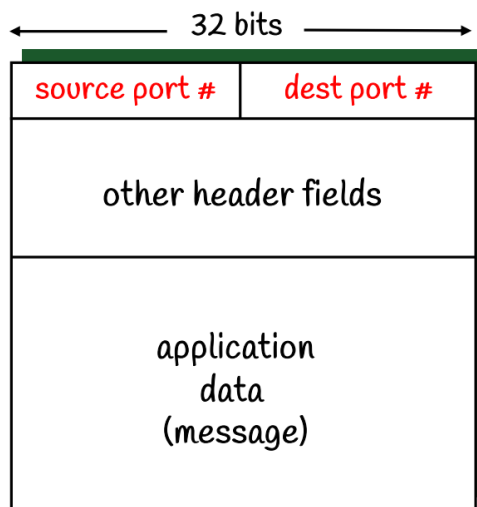
9.1 Types of Transport Protocol

- UDP
 - Data messaging service
 - No frills extension of best effort IP
 - The U stands for unreliable
- TCP
 - Reliable, in-order delivery
 - Requires setup
 - Has to discard corrupted packets
 - Needs to retransmit lost packets
 - Cannot overload receiver
 - Cannot overload network

9.2 Multiplexing / Demultiplexing

- Host receives IP datagrams
- Each datagram has a source and destination IP address
- Each datagram carries one transport-layer segment
- Each segment has source and destination port number
- Host uses IP addresses and the port numbers to direct the segment to the correct socket

A TCP/UDP segment format looks like this:



9.3 UDP “User Datagram Protocol”

Lightweight.

- No delay
- No connection state
- Just send it if I’m ready to send
- Smaller header overhead
- Thinks overhead of connection establishment is overkill
- Only contains IP and port numbers, and optional error checking which probably goes unused
- UDP header is only 8 bytes long (in image below, 4 bytes per row)

SRC port	DST port
checksum	length
Data	

Who uses it:

- Streaming – don't let TCP slow it down
- Phone calls, videos
- Simple queries such as DNS, where bi-directional communication isn't that required

9.4 TCP

- Requires an explicit connection.
- Sends and receives a stream of bytes, not message.
- Reliable and in-order, using checksums to detect corrupted data
- Uses sequence numbers to detect losses and re-order data

TCP can deal with reliability issues such as:

- Channels with some bit errors. Receiver must be able to detect error and ask for a repeat
- Receiver should detect errors

9.5 Sending Data VIA TCP

TCP sends data in segments, limited by the max segment size (MSS). A segment of data is sent when:

- It's full
- Not full but times out
- An app forces it

A TCP segment, inside an IP packet, consists of a header and its data. Hence, the MTU constraint of the IP packet **stacks**.

9.6 Sequence Numbers + ISN

- When a TCP packet is sent, it comes with a sequence number corresponding to the first byte.
- When the receiver sends an ACK, it sends the **next expected byte**, which can disambiguate what the receiver is talking about

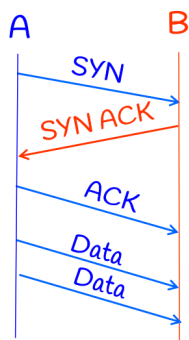
But when starting out, you can't say that the first byte's sequence number is 0, that's where the initial sequence number (ISN) comes out. There are so many ports and 65536 is not enough.

So, to establish a connection, hosts must exchange ISNs.

The ISN goes up to 2^{32} and ticks up by some amount every $4\mu s$. It takes 4.5 hours for the ISN to completely loop around, which I think it should be enough to disambiguate everything, right?

9.7 TCP 3-Way Handshake

A 3-way handshake is done to establish connection. Ideally, this happens:



Where:

- → SYN: This is my ISN, and I want to talk to you
- ← SYN ACK: it's me, and I want to talk
- → ACK: I've seen your SYN ACK
- → DATA: here's the data

A can talk to *B* as soon as it receives the SYN ACK from *B*

B can talk to *A* once it receives the ACK from *A* afterwards

The TCP header looks like this:

Flags: SYN FIN RST PSH URG ACK	Source port			Destination port		
	Sequence number					
	Acknowledgment					
	HdrLen	0	Flags	Advertised window		
	Checksum			Urgent pointer		
	Options (variable)					
	Data					

Deep diving into the packets:

- A's SYN:
 - Source port = A's port
 - Dest port = B's port
 - Seq number = A's ISN
 - Header length is 20
 - Flag is SYN
 - A tells B it wants to open a connection
- B's SYN ACK
 - B tells A it accepts and is ready to hear the next byte. When A receives it, A can start sending data
 - Source port = B's port
 - Dest port = A's port
 - Sequence number = B's ISN
 - Flags = SYN | ACK

9.7.1 When the SYN gets lost

- SYN lost $\Rightarrow$ SYN ACK does not arrive

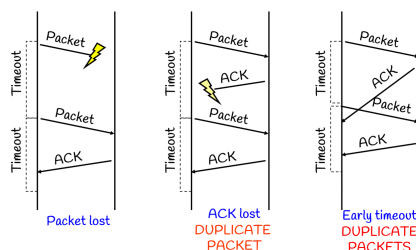
- A needs to set a timer and wait for the SYN-ACK and retransmit the SYN if needed
- Some TCPs default that timer to 3-6 seconds, but the user has the option to force it faster
 - User clicking reload makes browser create a new socket and do a connect, forcing a faster send of the SYN packet. Sometimes, very effective

9.7.2 Automatic Repeat Request (ARQ)

Receiver sends ACK when it receives packet

What if the senders' wait for ACK times out?

Just have the sender send it again.



What causes these retransmissions, where the sender needs to send another of the same packet?

1. Packet gets lost before it reaches the receiver
2. ACK gets lost
 - a. Results in a duplicate packet in the receiver
3. ACK reaches the sender too late
 - a. Even more duplicate packets

9.7.3 Trade-offs of Timing Out

- Too short: wasted transmissions

- Too long: excessive delays when packet lost

Timeout is variable. Therefore, TCP sends timeout as a function of the round-trip time (RTT). It expects ACK to arrive after an RTT. How do we know the RTT?

- Estimate the RTT based on previous data
- Maybe by a rolling average
- Where timeout will be double the estimate RTT

This brings a possible problem:

- Retransmissions can distort the RTT time. When retransmitting, basing the send time on the first attempt could result in a double RTT, and basing it on the second attempt could result in a halved RTT
- To solve this problem **do not factor in RTTs for packets sent more than once**
- This is the Karn/Partridge algorithm

9.8 TCP Sliding Window

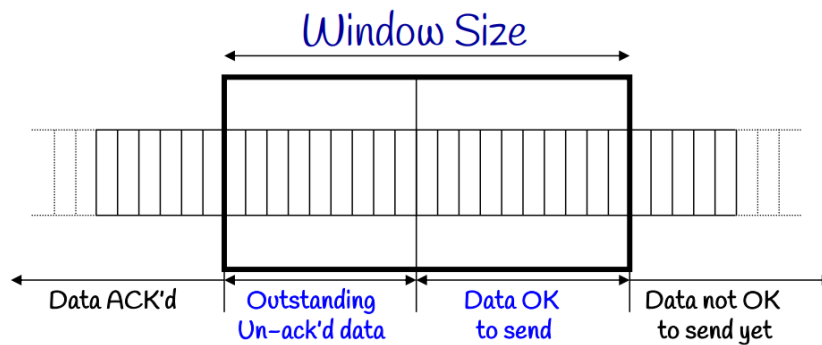
Stop and waiting for each ACK is terribly inefficient, since this causes only one TCP segment to be in-flight at a time. This makes sends extremely inefficient, since high-speed trips are limited by one packet per round-trip-time. This means that if the bandwidth is extremely high, the delay takes over.

This is the motivation for sliding window. It allows for larger data that is in flight, which allows the sender to get ahead of the receiver, but not too far ahead.

The sliding window is the amount of data that can be sent without an ACK.

The receiver needs to be able to store this amount of data.

Don't overwhelm the receiver! Don't create congestion in the network! Receiver advertises the window to the sender in the form "I cannot handle more than ___ bytes of un-ACK'd traffic."

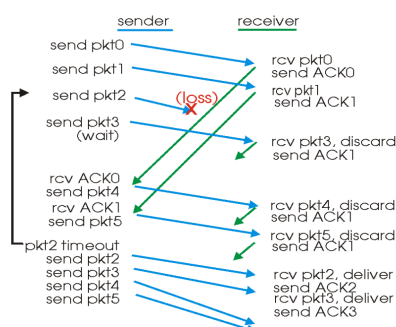


And that's where the window is advertised:

Source port		Destination port	
Sequence number			
Acknowledgment			
HdrLen	0	Flags	Advertised window
Checksum		Urgent pointer	
Options (variable)			
Data			

9.9 Fast Retransmission

When we can send multiple packets at once, what happens if just one packet is dropped?



- Sender sends packet 0
- Sender sends packet 1
- Sender sends packet 2

- Sender sends packet 3
- Sender receives ACK 0
- Sender sends packet 4
- Sender receives ACK 1
- Sender receives ACK 1
- Sender receives ACK 1

Oops, 1 hasn't come yet! Something's going to time out! And then when the timer expires the sender is going to have to retransmit all the way from packet 1 to packet 4 or onwards all again, which is pretty wasteful.

Here's how it works:

- If the sender gets 3 duplicate ACKs, send what was lost again
- The receiver might immediately jump the ACK to a higher number rather than $\text{lost} + 1$, which signifies that it has received everything between

This type of fast retransmit works for:

- Long data transfers
- High window size
- Low burstiness in packet losses

10 Congestion Control

We want high throughput for routers (bits / second of data that gets through), and we want low delay. However, too much data? This results in congestion collapse, where packets get dropped. It's just like any traffic situation.

The problem is, the more throughput, the more the delay. We want high throughput, low delay. What do we want? How do we define what we want?

Service time increases exponentially with load. This is how we model links in the internet. If you plot the average packet delay.

Small load, small delay.

As load gets increased, load is still small.

Closer to the capacity, the delay goes significantly up.

A simple metric of how well the network is performing, we defined a new parameter: power.

$$\frac{\text{load}}{\text{delay}}$$

As I increase the load, power goes up

After a certain point, power goes down.

TCP aims to maximize this power metric. Here, it is a simple ratio of load to delay. There are different ways to define this. $\frac{\text{load}^2}{\text{delay}}$. At least the default simple model this helps balance throughput. Just, which one do you care the more about?

We won't try to optimize this, but there are some algorithms that optimize for this metric. High throughput, low delay, there's another factor: fairness. Because we are serving different people, if I have two customers, I can give the 10GBps and the other one 0. Or I can give 5Gbps etc. Both have the same throughput, but one is fair, the other is not fair. The fairness is another metric to be careful about when dealing with congestion control.

Fairness is more complex than dividing the bandwidth.

Consider 3 flows.

10.1 Congestion Window Evolution

If an ACK is received:

$$W \leftarrow \frac{W + 1}{W}$$

In other words: increases by $\frac{1}{W}$, so increase gets smaller over time

Never go below one maximum segment size. Please don't go below the minimum.

If a packet is lost:

$$W \leftarrow \frac{W}{2}$$

So model:

$$\begin{aligned} 1 + \frac{1}{1} &= 2 \\ 2 + \frac{1}{2} &= 2.5 \\ 2.5 + \frac{1}{2.5} &= \dots \end{aligned}$$

As the congestion window size grows, the round trip size grows.

Need to start with a small congestion window size to avoid overloading the network.

If you want to use the full bandwidth of your router:

If a packet size is 10KB, and my router can send stuff at 100 Gb/s, then you can send as much as $\frac{100 \times 10^9}{10^4}$

Very unlikely for you to hit the maximum from this rate. A single packet drop halves your progress. How do we solve this?

In TCP, we add a slow start phase (really fast start).

Rather than increasing the CW by 1, we just double it.

So this prevents the linear increase from being slow.

Double CWND per RTT means increasing CWND by 1 for each ACK received.

On a loss:

- Triple dupe: /2 and keep going
- Timeout: whole bunch of packets lost, so better to start over with a slow CWND. Exponentially grow UNTIL half of what caused the timeout before. And then switch to linear.

10.2 Why are Deep Buffer Routers not good?

Apparently reducing buffer sizes didn't mess something up. Hence, large buffer sizes don't do any good.

10.3 Can't Change these

Different operating systems have very strange default values for the receiver congestion window size.

Too long ago windows had a limit of 16KB, linux had a limit of 64KB, very small. If the link caps are 100 Gbps, it's not enough to make the link busy all the time.

Why was the congestion window set to 16KB on windows? What was the reason? Why doesn't microsoft take the 16KB congestion window size and change it to 64KB?

Today's machine can handle more than that, the why did windows devices allow switching to a larger level?

They know that increasing the initial congestion window size is going to help, and they know their laptops can handle the receiver window size that is larger – they know that today's windows machine can handle.

The concerns – if they did it today, and tomorrow, the 16KB size was changed to 1MB. The day after, when the change is pushed to half the internet, suddenly, the dynamics of the network traffic is going to change significantly. It might create scenarios that operators will not be prepared or ready before. It's going to break things. It has nothing to do with Microsoft or windows. It is going to break things because nothing is prepared for it. Who is going to take the blame? Microsoft.

Because of that, they're extremely cautious.

Change is very difficult.

10.4 Repeating Slow Start After Idle Period

When the sender pauses and wants to resume, should I retain my congestion window size? The nature of the traffic in the network may change significantly. If I try to send at the same speed, I might congest the network. Hence, most TCP senders do a slow start restart. Go back to 1, exponentially grow to the exponential value's half, then grow linearly.

10.5 Nagle's Algorithm

Motivation?

Completely interactive applications.

The problem with interactive applications:

1. Type 3 characters
2. Press enter
3. TCP usually waits until the entire segment size

If TCP waits for me to type so many more characters, it is going to wait for a long time.

If, after I type 3 characters, it gets sent right away, but it causes a lot of overhead for non-interactive applications.

The question is – how can we balance the two scenarios? Nagle's algorithm is to do exactly that.

You want to limit the number of half-full packets you send. You want to send as full packets you can, but sometimes you want to avoid half-full packets so interactive applications can work.

So:

1. Wait if the data is small
2. If there is no data in flight, package and send
 - a. If no ack, no sending again, just wait

3. Therefore, at most one small packet per RTT. Can be send at a time.

So, how does this affect performance?

- Interactive applications: enables batching of bytes
- Bulk transfer: transmits in MSS-sized packets anyway

10.6 Motivation for Delayed ACK

In TCP, we've talked about the 3-way handshake. TCP is a bi-directional connection. Once a connection has been established, talking can be done.

Data travels in both directions, and ACKs travel in both directions.

How do we reduce the overhead of these ACKs? These packets have huge overhead. 40 bytes of header...

If you can piggyback ACKs with real data packets, this asves a lot of overhead.

Hence, TCP header allows piggybacking. You can use the same packet used to transfer data to ACK something else.

Suppose A and B talks to each other.

A sends to B

B receives. B must send back data and ACK at the same time, in the same packet.

No data? No need for Data + ACK, and probably send the data back later.

10.6.1 Where does delayed ACK come to play?

Sometimes, it might be worth to wait a bit to send back an ACK.

Maybe a task takes very short to do... in interactive applications... just give me 2ms to come up with the data for the `ls` command to complete.

So, let your app have 200 – 500 ms to generate data. Too long? Ack anyway.

And make sure you ACK every other full-sized packet (40-60 bytes, not just 3 bytes like “ls -l”).

Congestion is inevitable.

Congestion can be handled.

11 DCN

Machine learning has taken a lot of attention recently, and it impacts computer networks. Because ML applications are very bandwidth hungry, the networks need to adapt. Meanwhile, there are new tools and opportunities.

Data center networking:

- Design decisions
- Topology
- Transport

11.1 Data Center Network

What is a data center network? It is a collection of network, compute, and storage that is massive, all in one place. When the scale goes to 10s of millions of servers in one place, good things start to happen. The main motivation: if you have all these machines next to each other, certain tasks become easier. This is way better than having to compute across machines that are all over the world.

When you want to make a collection of servers, storage, and networks, the cost doesn't increase linearly. To have a location with 10 servers, it probably costs \$100K. To go from 10 machines to 100 machines doesn't multiply the cost by 10. It increases slightly less. If you go from 100 to 1000, the marginal cost is less. So, by increasing the size, you're trying to reduce the overall cost of the infrastructure.

Once you have all these resources in one place, you can use it for:

- Data storage and management
- Application hosting
- Data processing

11.2 How did data centers evolve?

Early data centers are very proprietary, where you can't access them. Until the 1990s, where the client-server model started gaining popularity. Virtualization started arriving in the 2000s.

In the early 2010s, we had cloud computing. Amazon started selling services to other companies. In the last 10 years, edge computing and IoTs gained popularity, and machine learning in the last 10 years started gaining a lot of attention.

11.3 Data Center Network Design Dimensions

- Cost? Is it expensive?
- How can networks scale?
- If you're in a location where real estate is very expensive (and that's why big companies move their data centers into specific locations)
- Efficiency: for one unit of energy I inject to the data center, how much extra energy do I need to maintain the data center? If the efficiency is 1.2, it means for every 1 unit of energy used for compute and actual work, it costs 0.2 units for cooling. The more efficient the data center, the closer it is to 1. Google has been doing a lot of work and has been leading the effort to reduce this number.
 - Take advantage that Canada is cold!
- Regulations: if you have some personal information in a data center, there may be laws that prohibit taking them outside of Canada. There are geopolitical issues and regulations that add regulations. The US certainly does not want all their credentials in other countries.

- Performance optimization: How do you balance the load? How do you ensure you have good quality among customers?
- How much are you using your infrastructure efficiently?

These, in general, are things you have to keep in mind when you are building data centers.

11.4 Architecture of a Data Center Network

Racks are “those” things you see inside office buildings (stereotypical image of a server).

On top of each rack is a switch that is connected to the servers. These are called access-layer switches, connected to each other using the aggregation/distribution layer. On top of that layer we have the core layer.

The core layer is the network layer. There is a router in the core. You see IP addresses, and every routing happens based on IP addresses.

In the aggregation/distribution layer, these are considered ethernet switches, so everything you’ve learned about ethernet applies here.

Of course, Mac addresses and the link layer do not scale. Ethernet has issues with scaling due to broadcasting. Hence, we add something in between. The aggregate devices operate both in the network layer and at the link layer. They do the translation, and they create separate local area networks. Below aggregation, we deal with link layer and ethernet. Above aggregation we deal with routers.

Usually there are 24-48 racks connected to aggregate switches. Links connecting aggregate to the core should have more capacity than to racks (usually a falloff by $\frac{1}{2}$, since most of communication happens inside the rack / clusters. There is locality. If you have multiple servers working together, many communications have locality).

11.4.1 Routing Traffic Within a rack

Routing within a rack is very fast.

If you have traffic between two racks, different things can happen.

- Sometimes, it goes to the top-of-rack switch, goes to aggregation, to the core, and back.
- Sometimes, it doesn't have to go directly back to the core.
- To talk to the outside world, core routers have connections to the outside world.

RTTs will be different in data centers. Talking to the outside world is the slowest.

This is called the 3-tiered architecture topology. Problem: It is unreliable. If the aggregation switch dies, all the racks connecting to that switch become broken. Hence, we have another approach: **fat-tree architecture**.

A pod is a collection of top-of-rack switches and aggregate switches.

Before, it was that each switch connected to one aggregate layer, but now, there is redundancy.

Each core switch connects to all pods.

This structure has a lot of reliability. There can be many different paths. When you reach a switch, you have the choice of which aggregate switch to go through. There are many paths with lots of redundancy. When you reach an aggregate switch, you have the option of which core to connect to. Once you go to a core, you don't have any options: you must come back down to a given path.

As the size of a pod grows, the number of paths becomes significantly larger. There could be 100 to millions more paths. Hence, it makes load balancing a very challenging problem.

This significantly helps with reliability. If a device goes down, use another path. If a link goes down, use another path. Hence, reliability goes up by a lot.

Fat-tree architecture is better because you don't need to have massively fast links. You just have more.

- Scalable
- Low latency high throughput

- Cost effective
- Multiply paths
- Automatic failover
- Better load balance
- Better redundancy
- Reliability and high availability and improved performance.

11.5 Other data center network topologies

With up to N nodes, where every pair of nodes are connected, you have $n(n-1)/2$ links. This is a perfect network in terms of performance. Any two nodes that want to talk to each other, there is no delay due to jumping. The problem is this is expensive. Any additional node means you have n more links to add, which is very costly.

A router has a limited number of ports. You can't add more than that. But in actual costs and practicality, it is the worst. So:

Notice how the no. of redundant paths go down as you traverse further down.

- Mesh
 - A link to every other node
 - Ideally best, but too expensive
- Two-tier
 - Tree with two layers. N spines, M leaves, everything connected like a neural network.
 - The maximum link length is 2, and there are lots of redundancy (N different paths per node)
- Hypercube

- Defined recursively. A 2D hypercube is a square. A 3D hypercube is a cube. You take two hypercubes of a lower dimension and connect **each corresponding** node.
- Maximum link length is $\lg(N)$.

11.6 Data center properties

- Extremely short RTTs
- Extremely high bandwidth links, i.e. very fast. Huge amounts of bandwidth available between two pairs of nodes.
- Extremely large transfers
- Single authority (typically)
 - No one else has to care about what goes on in my data center

These properties lead to challenges and opportunities.

11.6.1 ECN+DCTCP

A protocol done just for data centers.

DCTCP is a variant of TCP, using ECN as a congestion signal. Use -> reduced packet loss.

There is a sender, there is a receiver. You send packets, you receive acknowledgement. You keep a variable alpha. You mark this with ECN bits if things are being sent way too much.

You sent 10 packets, 2 of them are marked, so $\alpha = 0.2$.

You then adjust congestion window based on the extent of the congestion:

$Cwnd \leftarrow cwnd \times (1 - \alpha / 2)$

If alpha is 0.2, half of 0.2 is 0.1; hence $1 - 0.1$ is 0.9, so your new congestion window size is the old one times 0.9

Hence you're not reducing your congestion window by that much.

11.7 Link Layer Flow Control

Ensures that buffers are NEVER full.

There are two techniques.

11.7.1 Proactive

The transmitter gets some credit (any time you want, send me no. credit number of credits). The sender can use the credit without seeking permission. Immediately when I run out of credits, I have to stop. By doing that, the receiver controls how many packets they're receiving.

Once the receiver empties the buffer and serves the packets, it issues more credit. There is a tradeoff. You can issue a lot of credit at the start. The problem is that you can become overwhelmed. If there are 5 senders and you give them all 100 credits at once, might you not be able to handle all of them at once?

Making more credits available is making things faster but making things more risky. Less credits can degrade performance but are safer.

So, proactive is: you give credit before others.

11.7.2 Pause-Based

When your queue starts to build up, when you reach the pause threshold. You send a message to the transmitter, saying "stop". There is another threshold for resumes. In that case, send a message to the transmitter saying "resume". This is the most common link-level flow control mechanism.

Why is this good? It provides what we call a lossless connection. You don't need to drop packets. Before you had to drop packets, the "stop" message had to go through way too many hops.

There is a major challenge associated with this. When you are pausing, you are not pausing a single flow. You are pausing all the packets going to this buffer. Why is that a problem?

Imagine there are two flows.

One is very well-behaved. It sends one packet per second. The other one is very aggressive and sends a lot of traffic.

When the buffer becomes full, it stops it for everyone, which is effectively **collateral punishment**. This can cascade and bring down the entire network. You see, this can halt the entire network.

How do we address this? **By having multiple priority queues.**

You can pause individual flows, not every flow at once. That's the idea. This concentrates flows being paused instead of all of them.

11.8 Remote Direct memory Access

- Directly write to a remote server's memory
- RDMA-capable NIC handles data transfer entirely in hardware

REMOTE DIRECT MEMORY ACCESS: Inside a server, you can read and write from memory. It works, because there is no loss. If a network doesn't have loss, in RDMA, when two nodes want to establish a connection between them:

- Which areas of the local memory are going to be shared with the other side?
- After that, you don't need to involve the OS at all.
- The RNIC device takes over (Remote network interface card). This directly reads from local memory and writes directly to the remote memory. It sends the RNIC to the other side. The RNIC on the other side writes the data immediately. This bypasses the OS stack. This is much faster, and you can transfer huge amounts of data.
- Queue pairs (QPs): a send queue and a receive queue.

Benefits of RDMA:

- Low latency
 - Minimizes delays by avoiding CPU intervention
 - Ideal for applications requiring real-time data processing.
- High throughput
 - Enables faster data transfer rates
 - Suitable for HPC and large data sets.
- Reduced CPU load
 - Frees up CPU resources for other tasks
 - Improve overall system efficiency.
- Zero-copy data transfer
 - Eliminate (or minimize data copies)
- Applications: High-performance computing, storage

If you have multiple GPUs, how do you connect them?

RDMA used to be proprietary. But now you can use ethernet switches instead. You can have RDMA NICs and they work over regular ethernet.

12 Networks used in machine learning

Are those changes usable in machine learning requirements?

Let's look closely at machine learning workloads. For example, training a LLM. There are different models for LLM training. Data parallelism, model parallelism, or a hybrid between the two. What makes this different from regular data center traffic: it's going to be a mix of different types of application.

These mixes are going to be random. If you start watching netflix and I start watching youtube, there is a very different correlation between the traffic you generate and I generate.

If two people rent VMs and generate flows, there is going to be a very different correlation.

In ML applications, there are many correlations. You have large training tasks. There are dependencies between them. To compute something, you need all the data to be available. So, you'll wait for all the flows to arrive, then you will compute. Once you are done, you are going to spread the changes you've made, so multiple flows will start.

In other words, many people start watching the same thing at the same time.

On the internet, flows have different sizes. Here, flows are very large. Latency tolerance is very low.

ML workloads have extremely high requirements from the underlying network:

- High bandwidth
- Low latency
- Low jitter (variations in delay)

12.1 Moore's law limitation

One powerful CPU can solve very complex problems.

With two or more cores, not only do you need to compute in different processing nodes, you need to have them communicate.

The role of networking comes in the post moore's law era. We can't pack more transistors. We now need to pack cores. This is why networking has come back to the surface, and why everyone wants to make better networks.

12.2 What makes ML different?

Handling many independent flows in a network makes many network problems easier to solve.

In ML, we have few flows. Having few flows means each flow can have a large fraction of link bandwidth.

In ML, flows have direct/indirect dependencies.

Providing high-performance connectivity for ML workloads is extremely challenging.

For example, load balancing in ML.

Hence it is very important to know how to network things in ML.

12.3 Load-balancing

Incoming traffic to multiple output ports: randomly assign them to output ports. Because each flow is a small fraction of the overall bandwidth and because these are independent of each other, this works very well. Load balancing is completely random (equal cost multipath routing).

12.4 ...

Machine learning applications are a small fraction of what actually runs in a data center. The person who owns a data center wants to make money. Data centers are designed to support all sorts of applications. Because Machine Learning has gained so much attention and we are hitting the wall when it comes to Moore's law, it is essential to make networks better for machine learning.

ML applications have certain properties that make it challenging to provide networking for them. There are properties that make it easier. How can we use workload patterns in machine learning to adjust and adapt network topology?

Typical DCN have:

- Leaf spine
- Fat tree

Those are topologies that are built to work with all ranges of applications. When it comes to machine learning, we know the pattern of communication: GPU 1 sends this much traffic to GPU 2 at this point. If we have a reconfigurable topology, we can shift the topology just when it is needed.

This is how we do it in optical circuit switching. You can shift capacity between links to different nodes. How can we adapt the topology? There are very cheap, extremely simple mechanisms.

Physically: you can change mirrors to change connectivity of ports. These are very good in terms of cost since they are extremely cheap. They are very good at energy. The signal remains optical, the device is completely passive. But, the challenge is: reconfiguring these switches takes time. To move the mirror physically from one angle to another takes a few milliseconds. In a network, that is not acceptable. We want that to be done in the order of nanoseconds.

In ML, we don't need to change the topology every few nanoseconds. If we know that there is lots of traffic between one port to the other and we know before this traffic arrives, we can reconfigure and wait for the flow to arrive. This is an amazing alternative to traditional switches. If your wavelength is 40 GB/s of data, it works... This is independent of the rate, which is an amazing thing.

Another alternative is using active wave guides. When a light ray hits a prism, it reflects. Active waveguide changes wavelength. The signal comes in. If you turn it blue, it comes out at an angle. Turning it to green makes it go out of another angle. Changing the wavelength can switch the output ports the signal goes out. This is a very cheap mechanism, but it is timely to change the wavelength. In the ML context, because workloads are predictable, these are very powerful solutions.

We are not limited to topology. We are able to do other things. There is this thing called **application aware networking**. If we are aware of what an application wants, maybe we can adapt networks more intelligently.

The application is separate from the network layer, transport layer, and link layer. How-

ever, this operation has its drawbacks. The network does not know what applications want. If that's the case it can't optimize its operation. Fix? What if we provide some information from the application to the network? Application telling what it wants, requirements:

- Workload
- Characteristics of the traffic

If it can do that, the network can optimize its behavior. If the network knows the app will send 10GB of data in the next 2 seconds, it can provide a better route. Except the network does not have this information.

There has been a lot of recent research on this, and the suggestion is to create a new API between the application and network. Socket programming. What if we provide more flexibility to the application, to tell the network what it wants from them. What is the alternative? How can we solve the problem without burdening the application developer?

12.5 Incast Problem

A major problem in DCN is the incast problem. Incast problem is too many senders sending traffic towards one router at the same point of time. When it happens, it results in a microburst. There is a burst of traffic in a router. Routers and switches in the data center have small buffers. This results in many packet drops. Bursts are unpredictable, because we are looking at things from the network perspective. In the application layer, bursts are predictable.

Fan out, fan back. When a server sends a multi-get request to multiple servers, the same query goes to them. When they receive the query, they all respond back. That one request translates into many flows sent back. Because all of these responses are generated from that trigger, they fill up the buffer and cause many packet drops.

How can we solve this and make sure that we can change the routes, adjust the load balancing, and adjust the amount of buffering we have in the switches so that microbursts don't have that much of an impact?

The idea is, assuming we have access to the application. We don't need the app developer to do anything, but you need access to the source code.

You can profile the behavior of the app. Add a line to each function that logs when that function is called. When you call a certain function, you just log, "this function was called at this point of time." This system goes and logs network events, when there are a lot of packet drops. This is very easy. Make a system that logs individual function calls and also log what happens in the network. Then, you can correlate these two. You can create a graph, and you can figure out how likely it is to have packet drops when you call a particular function. After some milliseconds have passed, it is very likely that packet drops are going to happen.

Why is this important? If you want to predict when you will have a microburst in a network, the probability is uniform. But when you have analysis at the application level, you can tell, after 4ms that function is called, there is a very high chance a microburst occurs.

Once you identify that correlation, you can generate certain signals on the application side. A **smart tag** packet is sent. Once you see a smart tag, with 96% accuracy, you can tell, once you receive that smart packet, you will have a microburst 4ms later.

This lets the network tell the future.

When scaling the server count up, by default, lots of packet drops. When you use smart tags, you significantly reduce the packet drop rates. This is the power of integrating the application with machine learning. You can get information from the application side to make the network better.

Or you can use network information to make applications better.

12.6 Network Aware Applications

Why does TCP increase and decrease the cwnd? The network doesn't give these answers. TCP has to guess and increase and decrease the cwnd. What if the network could tell? That would be a very powerful mechanism. It can significantly enhance the performance of an application.

(not on the exam) In a DCN where you have multiple ML training jobs, you want to make sure that you can schedule these jobs (want to be able to tell, “job 1 has to start at $t=0$, job 2 has to start at $t=2$, and so on) to ensure that overlap is minimized. How do they do it? **They try to minimize the overlap.** Shift one of the jobs. Push the job back. Once you do that, you can add up the usage over time as a function and you’ll see that there are minimal conflicts. **This lets you schedule the jobs.** There are some serious limitations.

If these jobs don’t always last the same length, use the least common multiplier (LCM).

12.7 ML to enhance computer networks

So, how can we use ML to predict traffic? ML is very good at that. If you can observe repetitive patterns, traffic patterns over and over, and predict what happens next, you can make better decisions for routing and load-balancing.

There has been a lot of work in the recent years about replacing traditional heuristic-based protocols with ML solutions.

Maybe: use a RL agent, which tries to adjust the cwnd and observes what is the impact on the traffic throughput and the drop rate, and overtime learns how to improve the congestion control.

13 Overlay Networks

Overlay networks provide a logical view over a network. Instead of using a network’s physical connectivity, we consider certain views. We only focus on the application-level connectivity.

The links between applications are not physical links. They are logical links. Different nodes that are connected to each other, it’s a logical link. It is a logical network built on top of a physical network. You do not have a direct physical link between the two nodes. You have a logical link that is implemented as a tunnel.

You can have multiple overlay networks that share the same topology. Nodes are the end host of the network, and the reason we build overlay networks is about control. If you can create an overlay, if an application is written by you, you can control the application.

Routing overlays is the most common type of overlay network. When you have two nodes in a network (N1, N2), if you don't own the physical network, you have no choice on how traffic from one node goes to another. If you create an overlay network, you shift some of that decision to yourself.

One of the biggest problems is changing networks. For example, IPV4 and IPV6. Imagine you have some routers in the network, routers A,B,E,F that support IPV6 but you have two routers C,D that support IPV4 only. So what do you do instead? Create a tunnel. When A has an IPV6 packet to send to F, if you have a tunnel from B to E that bypasses the IPV4 routers:

- A sends an IPV6 packet to B.
- A and B both talk IPV6, everything is fine.
- C doesn't talk IPV6. But because we have a tunnel from B to E, every packet going from B to E goes to that tunnel. We encapsulate every packet at the start of the tunnel in an IPV4 packet and the destination is E. That's how you tunnel packets. You create a new packet (source is B, destination is E, and this is an IPV4 packet so C can understand it).

When a packet is for you, you decapsulate.

13.1 Tunneling in other layers

Encapsulate IP packets inside another IP packet.

Can be done in other layers, and in the wrong direction.

For example: TCP tunneling. Encapsulate IP packets inside a new TCP packet. There is a source, there is a destination. Source is sending a lot of IP packets. We decide to take

IP packets and put it inside another TCP packet, which goes in another IP packet. Why would we want to do this? Why might this be helpful?

Why?

- UDP looks sketchy. Let's wrap them in a TCP to bypass firewalls.
- Encapsulate regular IP packets and encrypt them, create another packet and transfer them.

13.2 What is TOR?

An overlay network used to ensure privacy.

Some servers are part of the TOR network.

When connecting to a webserver, you choose a bunch of nodes in the TOR network and send encrypted traffic through those nodes. You can have multiple layers of encryption.

Different nodes don't see the path.

13.3 Communication with Mobile Users

If you are streaming a video from [CNN.com](https://www.cnn.com), you are connecting from a gateway, and the gateway connects to CNN. If your IP address changes, the gateway IP address won't change. So, CNN will continue to stream data to the gateway. The gateway is aware that your IP address is changing. Hence, it will try to establish a new connection, adding something today's IP networks don't support.

Another example is Multicast. For Multicast, we deliver the same data to many receivers by avoiding sending the same data over and over. It is not very complex, but this requires collaboration between different routers.

13.4 ...

A few years ago, Adobe created a multicast overlay without you knowing it. In the agreement: *by using this software, you give permission from adobe to create an overlay network*. Many were not happy about this. Then, effectively you are paying for the cost of infrastructure that adobe needs.

13.5 Resilient Overlay Network

Imagine you have a network. You have a server, and you want to send traffic to somewhere else. Because you don't own the physical infrastructure, you don't make a decision on how your packets are routed, there is a chance that there is congestion in the path from one place to another. You still have one overlay decision. You can send traffic to someone else and explicitly ask them to forward the traffic. RON was a very successful project that shows that by creating an overlay network, you can reroute traffic without having to create a physical network.

Of course, it has its own overheads. You are encapsulating and adding delays, adding extra processing time. In some scenarios, it makes sense.

13.6 P2P

Napster was an application made by a freshman in northeastern university. The idea is: you want to share music files with others. The way napster worked, was:

- You have a client (the napster client you install on your machine)
- There is a directory on your machine. When you install a client, you point your app to that directory. That would be your music folder.
- The application goes and indexes the files over there, and then sends a copy of the names in that directory to a centralized directory. This is a database of which tracks are owned by who.

- If somebody else looks for a particular track, they will connect to the directory and ask who has this track. The directory responds back: *this node has X*, and you can download the music track directory.
- The idea: *I'm not sharing the files, I'm just helping people find each other. This can't be illegal, right? I don't even see the files! Someone tells me, I have these files. In other words, loopholes.*
- WRONG! Just 4-5 months after the company was founded, the first lawsuits occurred. Despite those lawsuits, in 2000, Napster reached 80M users. In Mid 2001, it went out of business due to these lawsuits.

Napster led to a huge number of new solutions, dozens of P2P alternatives. It started a movement.

Napster was then reconstituted as something completely different.

How would you loophole the centralized directory? You needed to tell people where to go. How do you remove the directory and provide the same service?

Have everyone hold their own centralized directory. If they are going to attack the centralized directory, what if we don't even have one? What if we don't have the network? A centralized directory is the point of failure - both physically (bottleneck) and legally.

We allow nodes to create an overlay network. The list of files is not shared. Everyone knows which files.

If you are looking for a new file, there is no centralized directory to tell you where to download the file. So, what do you do? You just have to ask. That was the idea behind what came after napster, which happened to be almost a year after.

Gnutella: you had an overlay network. You join by contacting a few nodes in the network. You will not be sharing the files. You are only connected. When you want to search for something, all you need to do is ask your neighbors. If your neighbors have that file, they will share it. If not, they will recursively ask their neighbors. You have removed the centralized directory and replaced it with a graph-search. Once you find someone who has the file, you directly contact them.

What is the problem with this? People acting malicious? Is this a perfect system if you had this today? Is it inefficient? Every time you need a file, you will have to ask your neighbors, behind links that might as well be slow. In 2000, many people were behind dialup connections. Your modem might have a max of 48 kilobits per second.

And then you have to ask their neighbors and so on, and this repeats. It is going to be very difficult to find the correct files.

How do you solve this problem? Innovation. Search is very bad on this network. How do you make it more efficient?

On a query hit, you will know who has the file. But this is very inefficient as you are flooding the entire network with your query, especially if you are looking for a rare file only a few people in the world have. How would you solve this problem?

The next generation tries to solve this problem. The idea is: at the start you join the network. But, KaZaA started to distinguish between nodes. Not everyone is the same. They created this notion of supernodes. There are nodes that have high stability, high reliability. They have high bandwidth and a lot of processing power. And there are nodes that don't. The stable nodes are called supernodes.

When you join, you join as an ordinary node. If you are reliable, you are promoted to a super node. You can use super nodes to act like the central server. Each super nodes have very good connectivity. When you connect to one of these super nodes, you are going to upload your directory to that super node. Just to your super node. If someone searches, they don't have to go through every node in the world. They just have to search within the super nodes. This significantly improves the search time. This was very successful. It's a very powerful mechanism.

It is easy to identify supernodes, but it is hard to find out who is hosting the supernode and threaten them to go down.

We make the search problem much faster. But there is still the download side. Once you find something, you have to download to someone who has the file and start downloading. The supernode has the directory, but it doesn't have the individual file. Hence, the actual downloading can take a long time.

So, what can you do about it? How can you fix the problem? These files can be very

large. How can you solve this problem?

13.7 Bit-you-know-what-this-is

Break the large file into small pieces.

You make sure the integrity of those pieces is preserved. Why? You don't need to take the entire file from one person. You can take chunk 1 from one person, chunk 2 from others, and so on. You can parallelize it. If the upload rate for your modem is very low, if you download from 10 different people, you are going to have 240 kbps download. More interestingly, as soon as you have one of the pieces, you not only can focus on other parts, you can become an uploader. Other people can start downloading that piece from you. That significantly accelerates things.

Components:

- Seed: peer with the complete file, fragmented in pieces
- Leacher: peer with an incomplete copy of a file
- "The file": Passive component, stores summaries of the pieces to allow peers to verify tier integrity
- Tracker: Allow peers to find each other. Returns a list of random peers.

Trackers are very lightweight and are very distributed. If you bring down one tracker, another one goes up.

How do you start downloading:

- Start finding that file.
- With that file, connect to a tracker to know who else you can find.
- Tracker gives you a list.
- If there are peers, you shake hands with the people who have the files.

Once you receive those pieces, you will start sharing. You will update and ask the tracker, saying now you have these pieces.

That makes downloading much faster. There are still issues (legally), people who go after whoever is using it, they still have their ways.

So, is this perfect? Is there a problem left?